

Study Plan
Practice
Revise
Succeed!

A Complete
Handwritten Notes

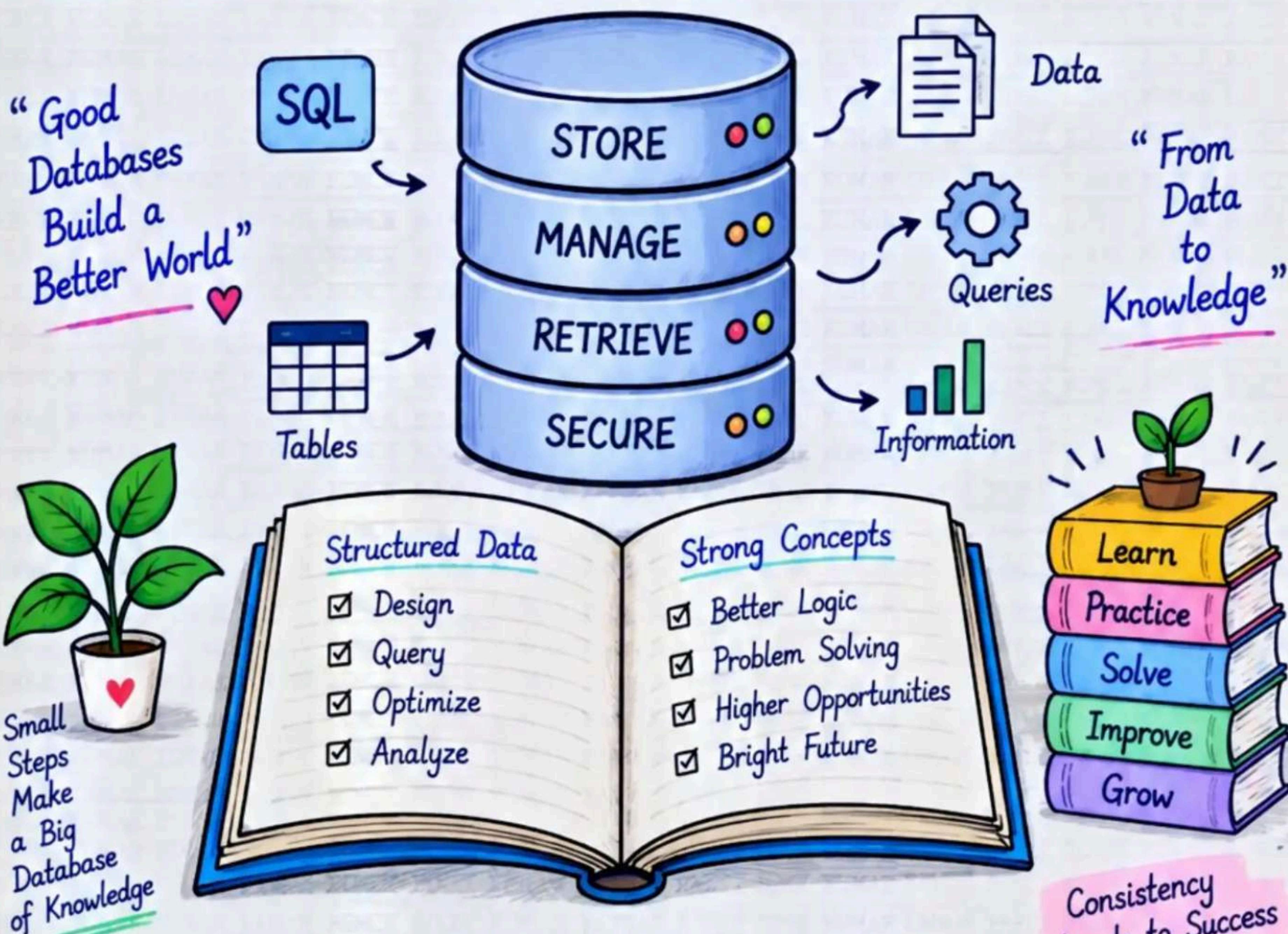
"Data Today
Better Tomorrow"

- ☒ GATE
- ☒ Semester
- ☒ Placements
- ☒ Interviews
- ☒ Self Study

DBMS

Database Management System

• Concepts • Diagrams • SQL • Numericals • GATE Qs • Interview Qs



Chapter Index



Database Management System (DBMS)

"Small Steps Make a Big Database of Knowledge" ❤️

- ✓ Study
- ✓ Practice
- ✓ Revise
- ✓ Succeed !

Ch.	Chapter Name	Pages	Ch.	Chapter Name	Pages
1	Introduction to DBMS	3 = 9	18	Triggers	133 = 138
2	DBMS Architecture & Data Abstraction	10 = 14	19	Cursors	139 = 143
3	Data Models	15 = 18	20	Transactions & ACID Properties	144 = 151
4	ER Model & ER Diagrams	19 = 27	21	Concurrency Control	152 = 160
5	Relational Model	28 = 33	22	Locking Protocols & Deadlocks	161 = 168
6	Keys & Constraints	34 = 37	23	Database Recovery	169 = 176
7	ER to Relational Mapping	38 = 41	24	File Organization	177 = 183
8	Relational Algebra	42 = 53	25	Hashing & Hash Indexing	184 = 190
9	Relational Calculus	54 = 59	26	B-Tree & B+ Tree	191 = 198
10	Functional Dependency	60 = 67	27	Query Processing & Optimization	199 = 206
11	Normalization	68 = 82	28	Distributed Database	207 = 213
12	SQL Fundamentals, Set Operations & Subqueries	83 = 97	29	NoSQL Databases	214 = 219
13	SQL Joins	98 = 108	30	OLTP, OLAP & Data Warehousing	220 = 226
14	SQL Functions	109 = 114	31	CAP Theorem & Advanced DBMS	227 = 232
15	Views in DBMS	115 = 119	32	DBMS Interview + SQL Practice	233 = 239
16	Indexes	120 = 126	33	Complete DBMS GATE Revision	240 = 246
17	Stored Procedures & Functions	127 = 132	34	Complete DBMS Mind Map	247 = 250

“

A well organized index today makes revision easier



Note :

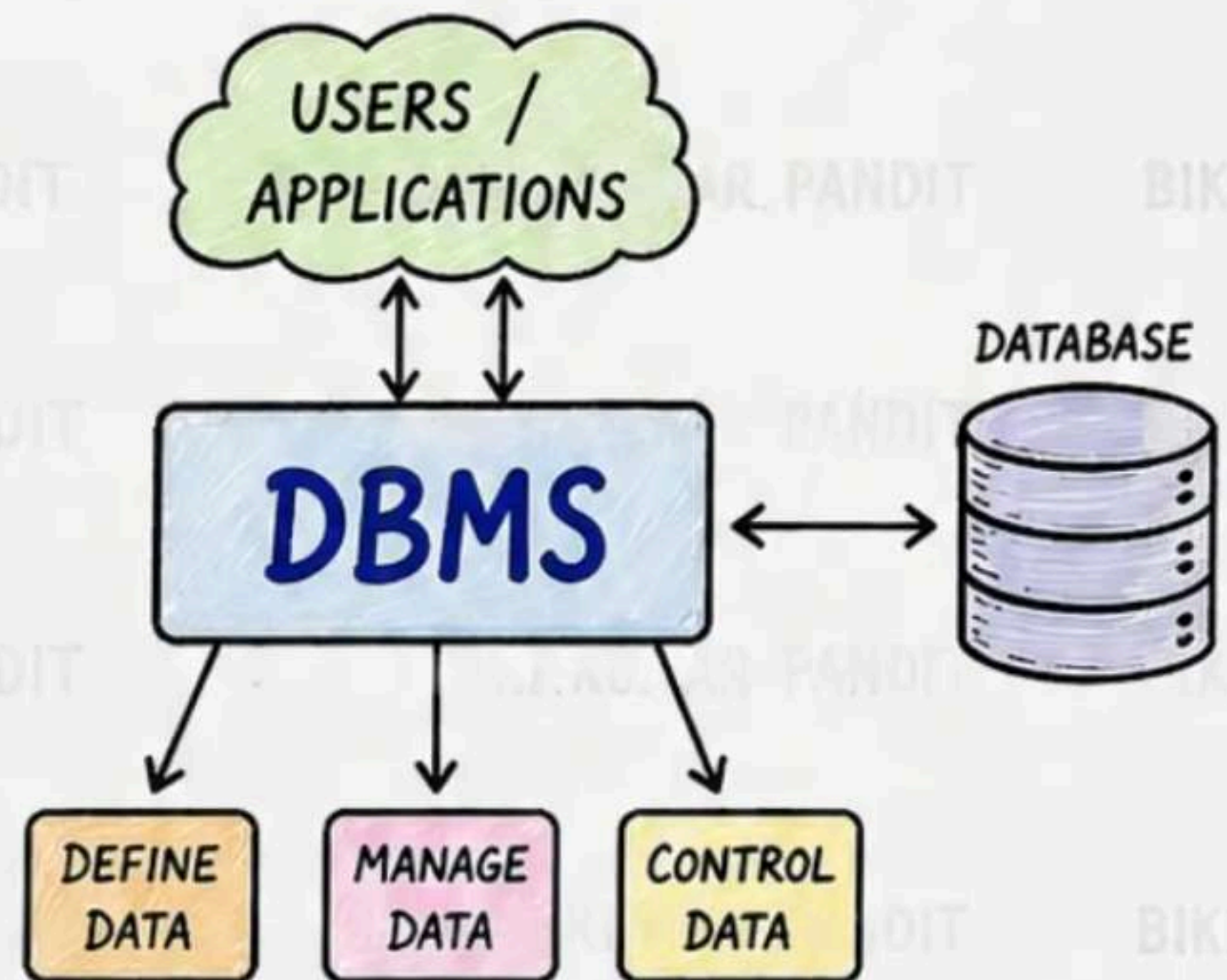
- ★ Page numbers are as per this handwritten notes series.
- ★ Important topics for GATE are marked

★ Chapter 1 ★

INTRODUCTION TO DBMS

★ What is DBMS ?

A Database Management System (DBMS) is a software system that allows users to define, create, maintain and control a database. It acts as an interface between users and the database, ensuring data is stored, organized and retrieved efficiently and securely.



★ Purpose of DBMS

- ✓ To store and manage large amount of data.
- ✓ To provide easy and efficient access to data.
- ✓ To ensure data accuracy and consistency.
- ✓ To provide security and data integrity.
- ✓ To handle data sharing and concurrency.
- ✓ To provide backup and recovery of data.

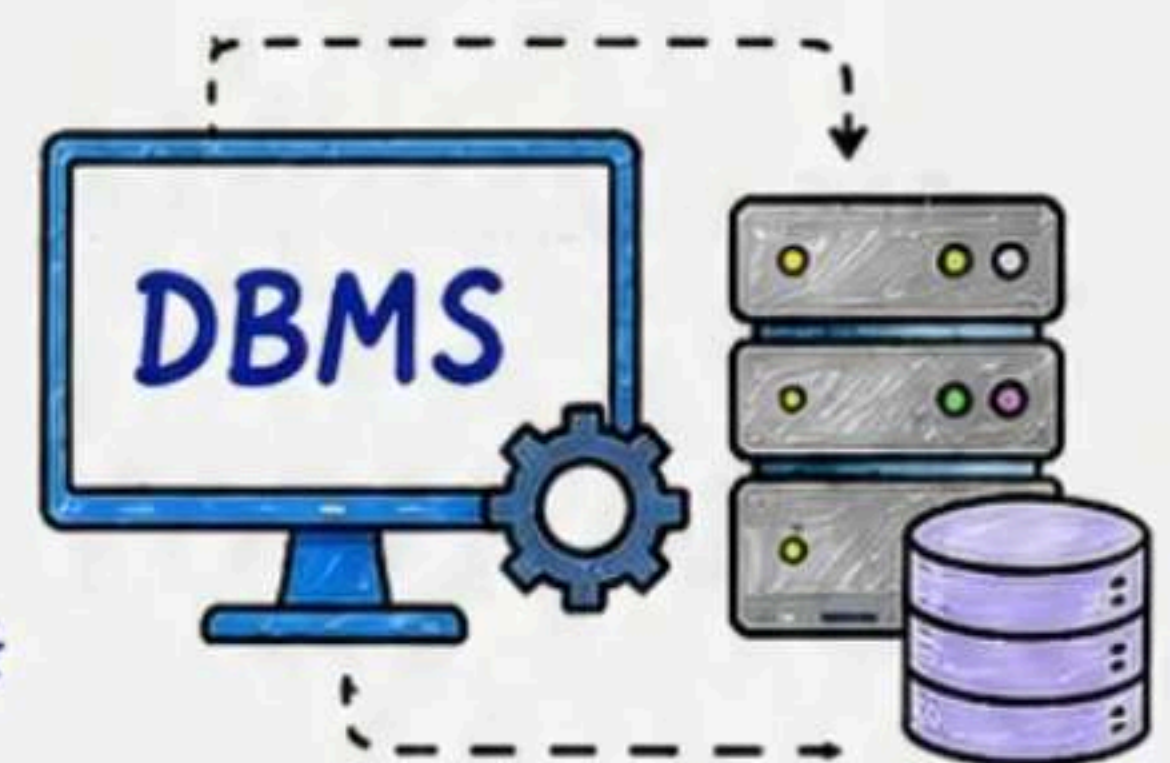


In Simple Words

DBMS is like a bridge between the user and the data. It helps us to store data in an organized way and use it whenever needed.

★ Characteristics of DBMS

- | | |
|-------------------------------------|-------------------------|
| ★ Self-Describing Nature (Metadata) | ★ Security |
| ★ Data Independence | ★ Concurrency Control |
| ★ Data Abstraction | ★ Backup and Recovery |
| ★ Data Sharing | ★ Multiple User Support |
| ★ Data Integrity | |

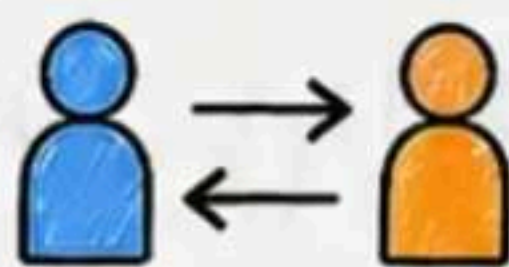


★ Advantages of DBMS



Security

Protects data from unauthorized access.



Data Sharing

Multiple users can share data easily.



Data Accuracy

Reduces redundancy and inconsistency.



Backup & Recovery

Helps to restore data in case of failure.



Efficiency

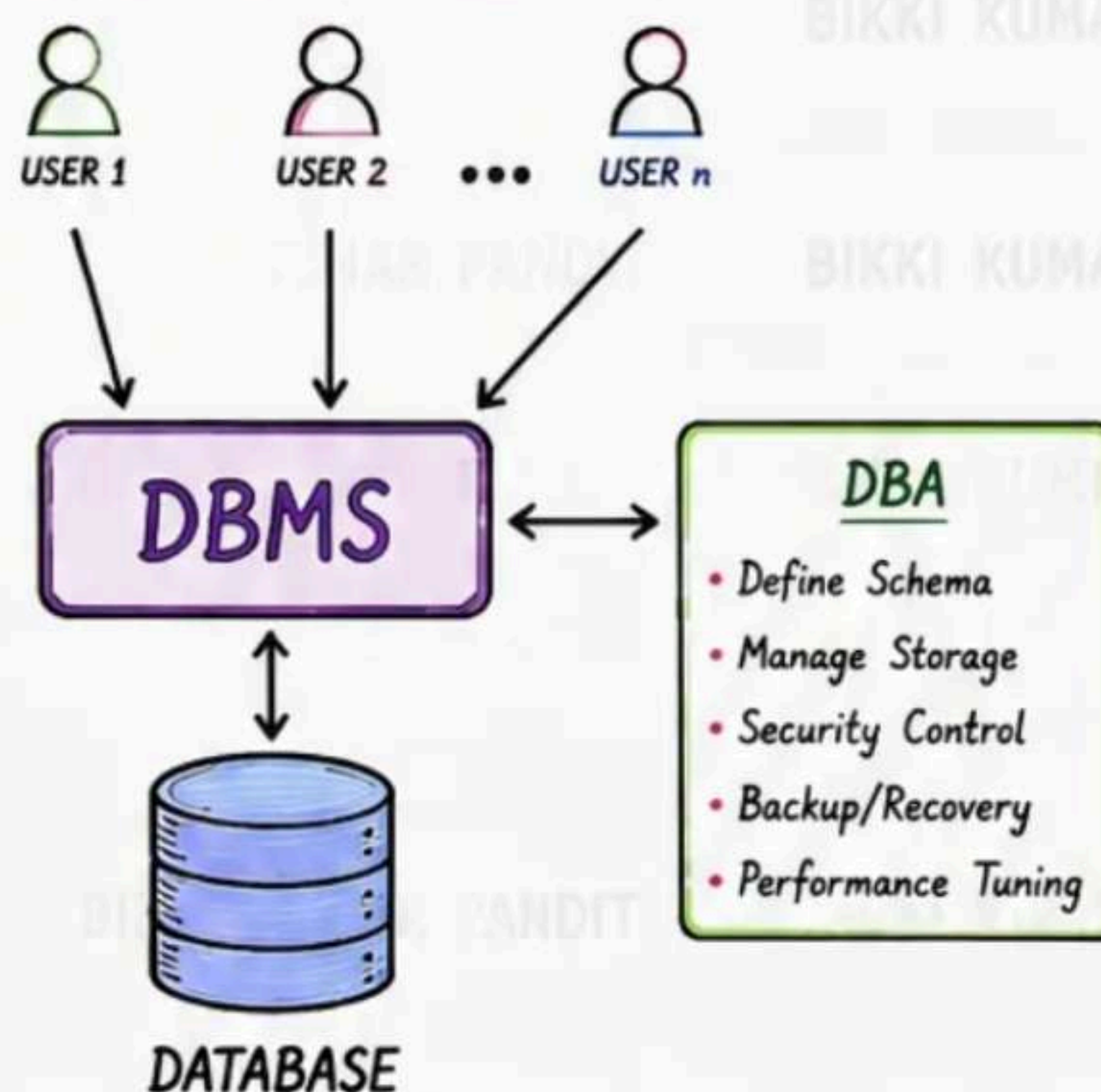
Improves productivity and data access speed.

★ What DBMS Does ?

DBMS performs the following major functions -

- **Data Storage** : Stores data in a structured and organized manner.
- **Data Retrieval** : Allows users to retrieve data easily and efficiently.
- **Data Manipulation** : Enables insertion, update, deletion and modification of data.
- **Data Security** : Protects data from unauthorized access and ensures privacy.
- **Data Integrity** : Ensures accuracy, consistency and reliability of data.
- **Concurrency Control** : Manages simultaneous access by multiple users.
- **Backup & Recovery** : Provides backup of data and recovers it in case of failure.

Basic DBMS Environment



Remember !

DBMS is not just about storing data, it is about managing data intelligently.

★ DBMS vs File System

Feature	File System	DBMS
Data Organization	Data is stored as files.	Data is stored in tables with relationships.
Data Redundancy	High (Duplicate data exists)	Low (Redundancy is minimized)
Data Consistency	Difficult to maintain	Easy to maintain
Data Sharing	Limited	High
Security	Weak	Strong (User authentication, authorization)
Concurrency Control	Not Possible	Possible
Backup & Recovery	Complicated	Easy and automatic
Data Integrity	Hard to ensure	Ensured by constraints
Data Access	Program-Data Dependence	Data Independence
Query Language	No standard query language	Uses SQL (Structured Query Language)

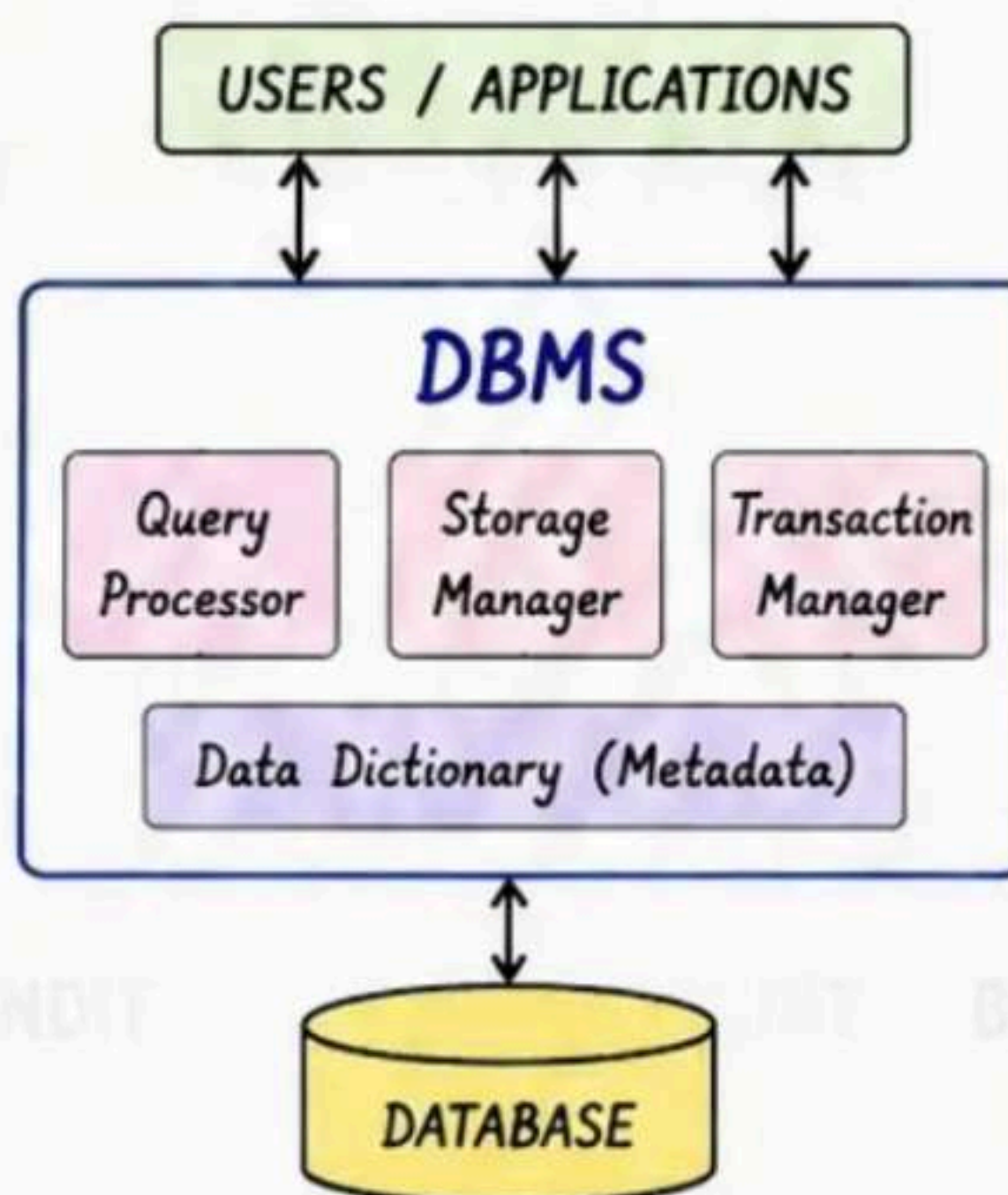
★ Examples of DBMS



★ Components of DBMS

1. **Hardware** : Physical devices used to store and process data.
2. **Software** : DBMS software that manages the database.
3. **Data** : The actual data stored in the database.
4. **Procedures** : Rules and instructions for designing and using the database.
5. **People** : Users, DBAs, Programmers, End-users.

★ DBMS Architecture (Simplified)



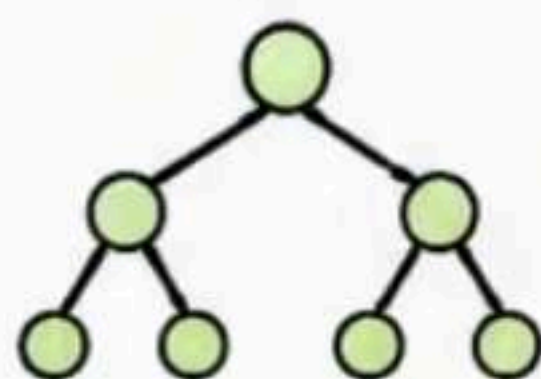
Note :



Metadata is data about data. It describes the structure of the database.

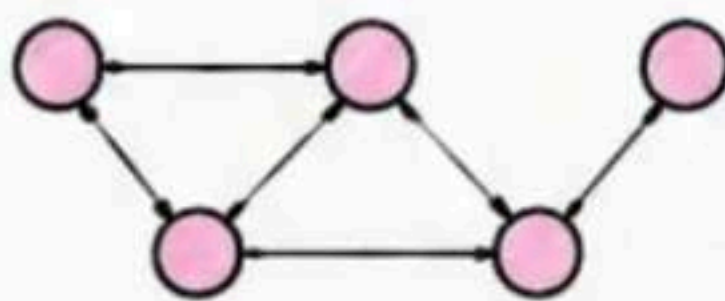
★ Types of DBMS

1. Hierarchical DBMS



Data is organized in a tree-like structure.

2. Network DBMS



Data is organized as a graph with multiple relationships.

3. Relational DBMS



Data is stored in tables (rows and columns) and related using keys.

4. Object-Oriented DBMS



Data is stored as objects which combine data and methods.

★ Data vs Information

RAW DATA

Unprocessed facts and figures.



PROCESSING

Data is processed and organized.



INFORMATION

Processed data that is meaningful and useful.



DECISION

Helps in making better decisions.

★ DBMS Languages

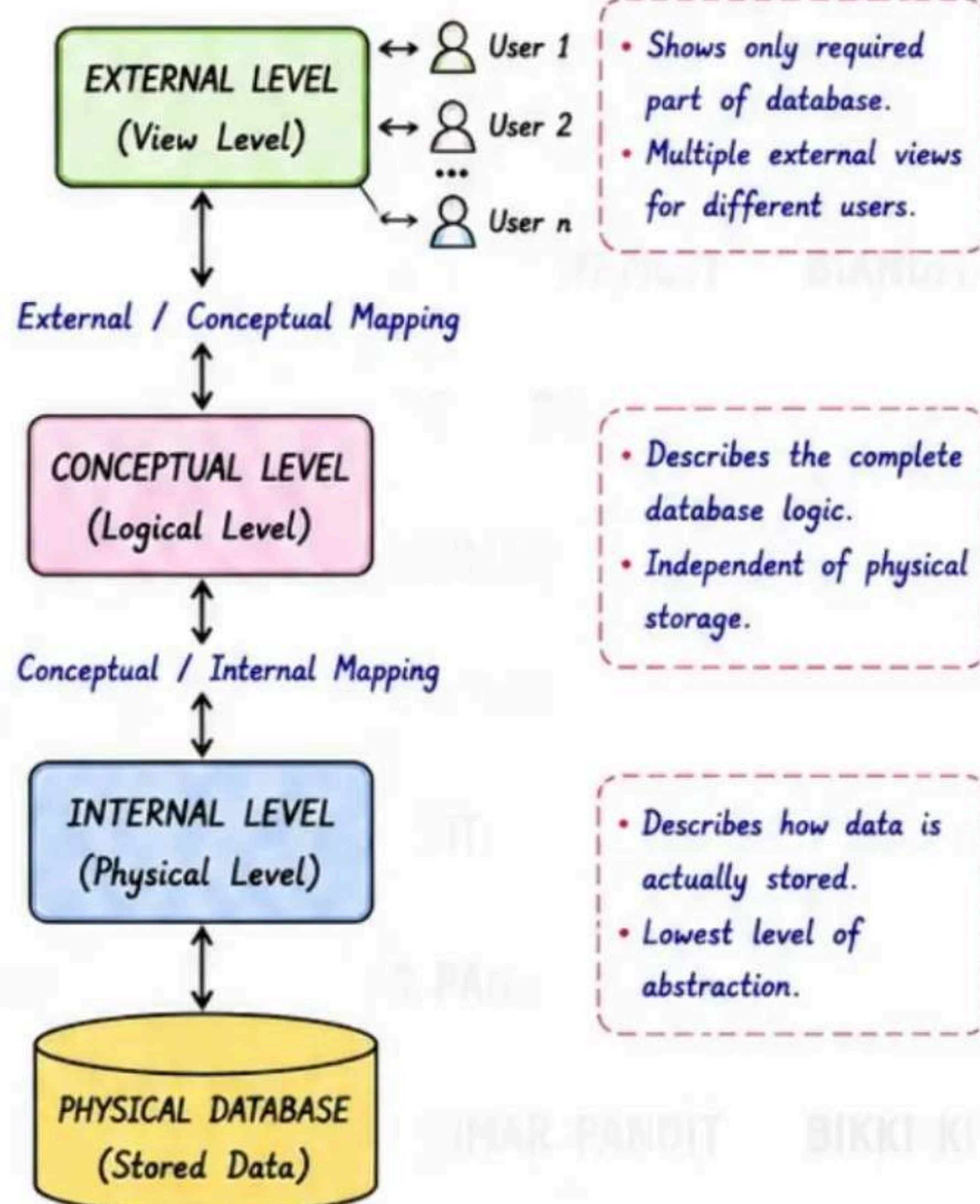
- **DDL (Data Definition Language)** : Used to define and modify the structure of database objects.
Examples : CREATE, ALTER, DROP, TRUNCATE
- **DML (Data Manipulation Language)** : Used to manipulate data stored in the database.
Examples : INSERT, UPDATE, DELETE, SELECT
- **DCL (Data Control Language)** : Used to control access and permissions.
Examples : GRANT, REVOKE
- **TCL (Transaction Control Language)** : Used to manage transactions.
Examples : COMMIT, ROLLBACK, SAVEPOINT

Language Type	Purpose
DDL	Define and modify database schema (structure).
DML	Insert, update, delete and retrieve data.
DCL	Control user access and permissions.
TCL	Manage transactions to ensure data consistency.

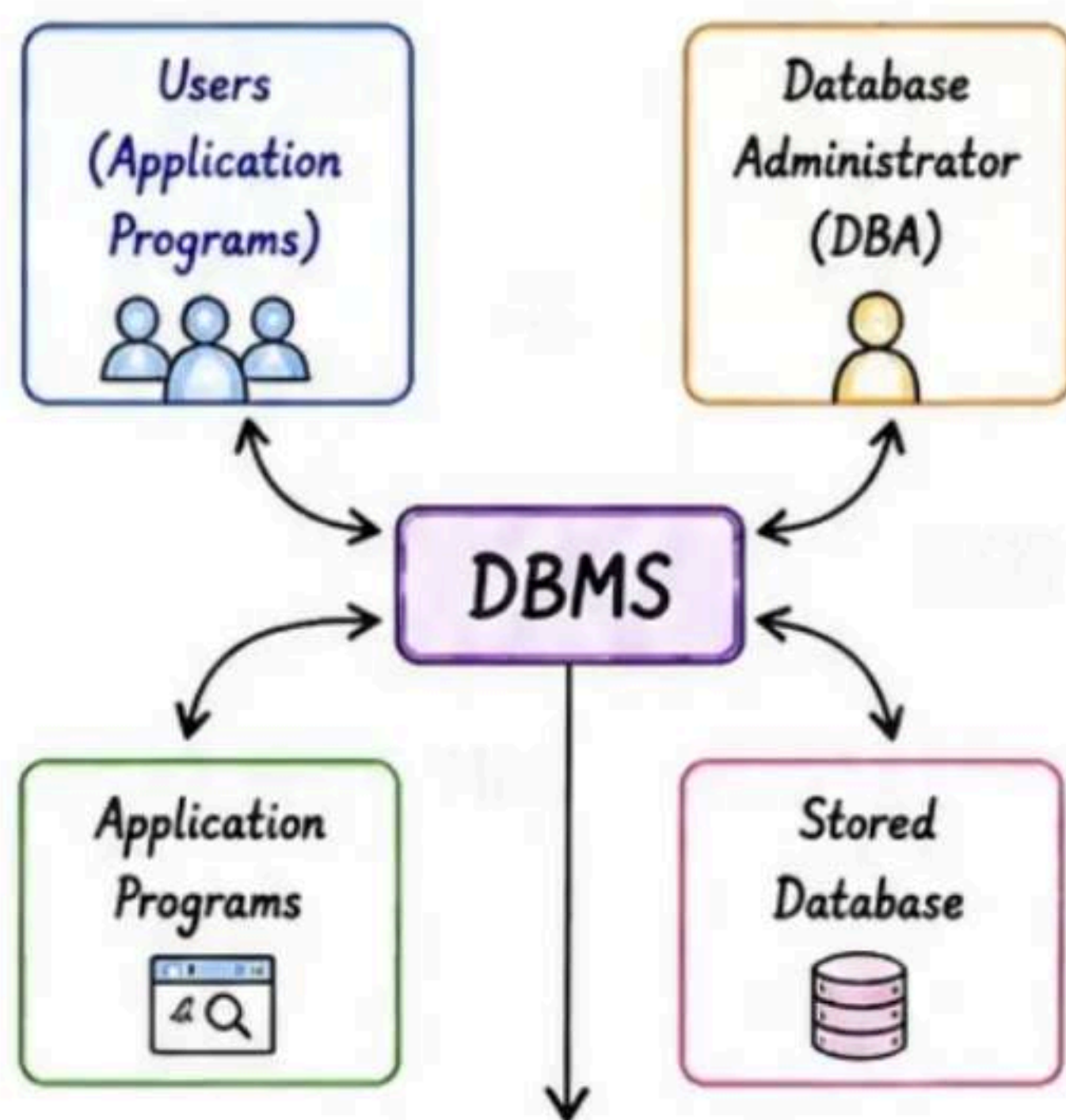
★ Database vs DBMS

DATABASE	DBMS
→ Collection of related data.	Software used to define, create, manage and control the database.
→ Data is stored in an organized manner.	Acts as an interface between user and database.
→ Generally represents a particular enterprise.	It can manage multiple databases.
→ Less intelligence.	More intelligence and features.
→ Example : College Database, Library Database.	Examples : MySQL, Oracle, SQL Server, PostgreSQL.

★ Three Schema Architecture



★ DBMS System Environment



★ DBMS Advantages

- ✓ Controls data redundancy.
- ✓ Improves data integrity and consistency.
- ✓ Provides data security.
- ✓ Enables data sharing among multiple users.
- ✓ Provides backup and recovery facilities.
- ✓ Offers data independence (logical & physical).
- ✓ Improves data access and query processing.
- ✓ Reduces development and maintenance time.



★ Limitations of DBMS

- ✗ High cost of software and hardware.
- ✗ Requires skilled and trained personnel.
- ✗ Complexity in design and operation.
- ✗ Greater impact of failure.
- ✗ Requires more memory and storage.

Example

Consider a school. It needs to store information about students, teachers, subjects, and marks. Using DBMS, we can store all this data in an organized manner, retrieve it easily, update when needed, and keep it secure from unauthorized access.



- DBMS is a software system used to store and manage data efficiently.
- It acts as an interface between users and the database.



★ What is a Database ?

A database is a structured collection of related data that is stored in a computer system and can be easily accessed, managed, updated and organized.



- Organized
- Structured
- Related
- Stored Electronically
- Shared
- Easy to Access

★ Why Do We Need DBMS ?

- ✓ To store large amount of data.
- ✓ To reduce data redundancy.
- ✓ To maintain data consistency and integrity.
- ✓ To provide security and access control.
- ✓ To allow data sharing among multiple users.
- ✓ To provide backup and recovery.
- ✓ To ensure data independence.
- ✓ To allow efficient data retrieval.



Remember !

Without DBMS, it becomes very difficult to manage large data manually or using traditional file systems.

★ DBMS vs Traditional File System

Feature	Traditional File System	DBMS
Data Redundancy	High	Low
Data Consistency	Difficult to maintain	Easy to maintain
Data Sharing	Limited	High
Data Security	Weak	Strong
Data Integrity	Hard to ensure	Ensured using constraints
Data Independence	Not Available	Logical & Physical Independence
Backup & Recovery	Difficult	Easy and automatic
Data Access	Program dependent	Data independent
Query Language	Not Available	Uses SQL (Standard Language)

Example

In a college,
Traditional File System :
Separate files for
Students, Courses,
Marks, etc.
(Hard to manage)

DBMS :

All data stored in
organized tables and
can be easily managed,
queried and updated.

★ DBMS Users

Database Administrator (DBA)



Manages the overall database system, defines schema, controls access, handles backup and recovery.

End Users



Use the database through applications to perform their day-to-day work.

Application Programmers



Write programs and applications that access the database using APIs / SQL.

System Analysts



Analyze requirements and design database systems to meet business needs.

Specialized Users



Use the database for specific applications like CAD, AI, Data Mining, etc.

★ DBMS Evolution

★ Characteristics of DBMS

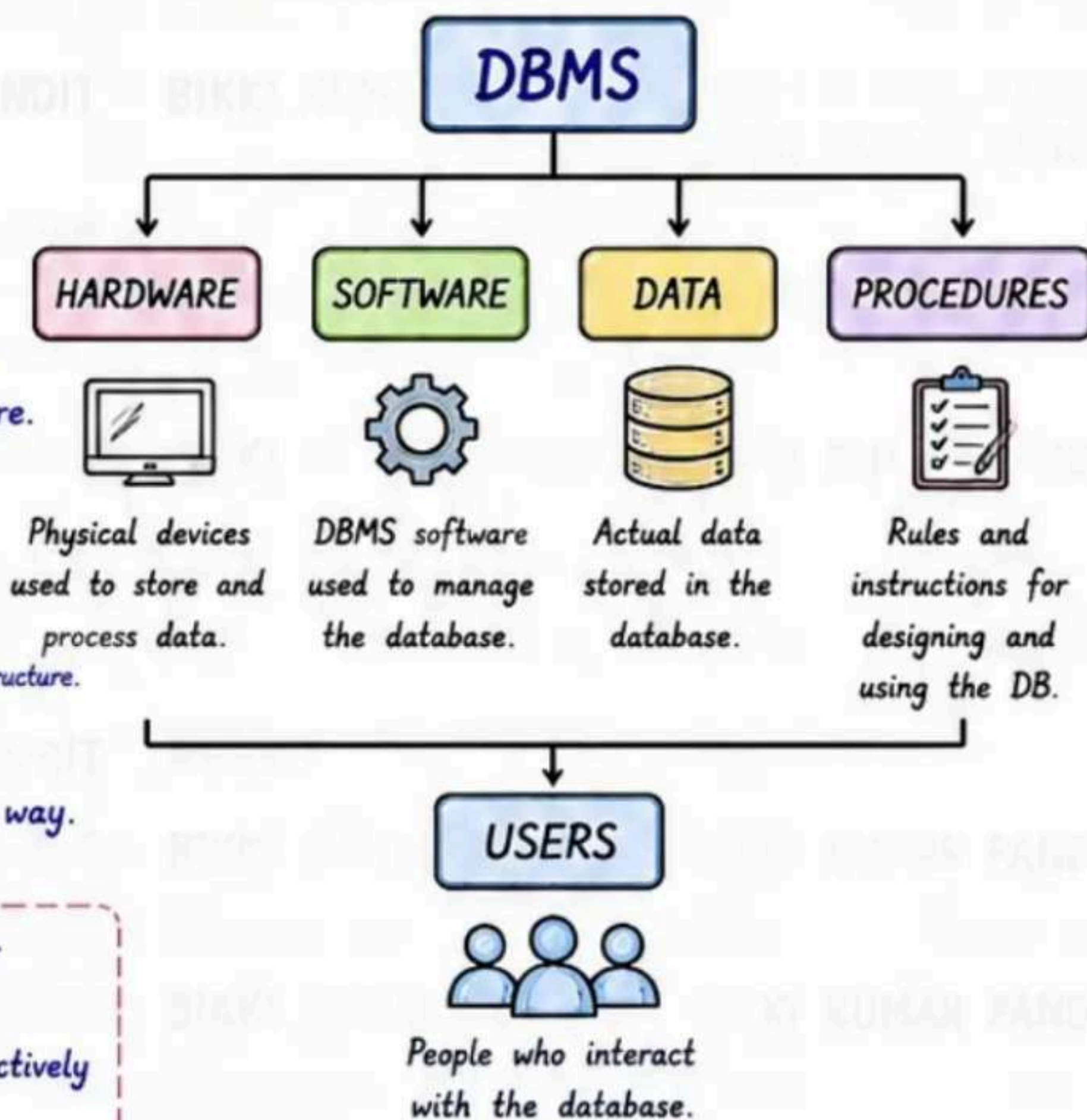
- **Data Independence** : Changes in one level do not affect other levels.
- **Data Consistency** : Ensures that the database remains accurate and reliable.
- **Data Integrity** : Maintains accuracy and validity of data.
- **Data Security** : Protects data from unauthorized access and misuse.
- **Data Sharing** : Data can be shared among multiple users.
- **Concurrency Control** : Manages simultaneous access by multiple users.
- **Backup & Recovery** : Provides mechanisms to back up data and recover in case of failure.
- **Reduced Redundancy** : Minimizes duplicate data and saves storage space.
- **Program-Data Independence** : Application programs are not affected by changes in data structure.
- **High Level of Abstraction** : Hides complex details and presents data in a simple way.

In Simple Words



DBMS is not just about storing data, it provides a controlled environment with various features that make data handling simple, secure, consistent and efficient.

★ DBMS Components



Note

- These characteristics make DBMS superior over traditional file systems.
- DBMS helps organizations to manage data effectively and supports decision making.

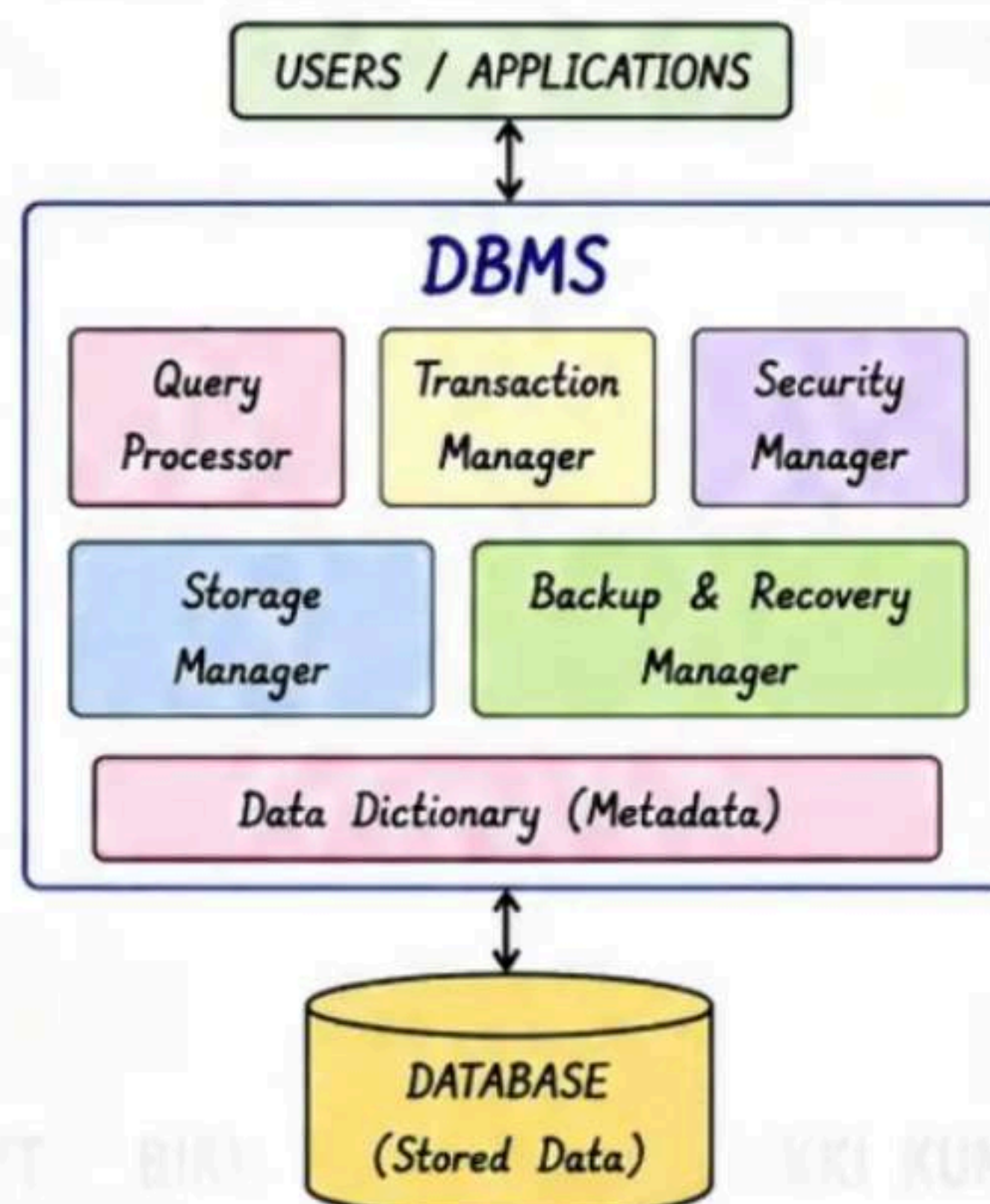
★ DBMS Applications

Application Area	Description	Examples
 Banking	Used to manage customer details, accounts, transactions, loans, etc.	Account Management System, ATM System, Internet Banking
 Education	Stores student information, courses, marks, attendance, fees, etc.	Student Information System, Library Management System
 Healthcare	Manages patient records, appointments, medical history, billing, etc.	Hospital Management System, Electronic Health Records (EHR)
 E-Commerce	Handles product details, orders, customers, payments, inventory, etc.	Amazon, Flipkart, Online Shopping Systems
 Travel & Tourism	Manages bookings, schedules, customer details, payments, etc.	Airline Reservation System, Hotel Booking System
 Government	Stores citizen records, taxes, licenses, voting, public distribution, etc.	Aadhaar System, Income Tax System, Ration Card System
 Manufacturing	Manages inventory, production, suppliers, employees, sales, etc.	Inventory Management System, Supply Chain Management
 Telecommunication	Handles call records, billing, customers, network data, etc.	Call Detail Record (CDR), Billing System

★ Database Components in Detail

- **Data** : The actual information stored in the database.
- **Metadata (Data Dictionary)** : Data about data. It stores definition of data, schema, relationships, constraints, etc.
- **DBMS Engine** : Core software that processes queries, manages storage, ensures security and concurrency.
- **Storage Manager** : Handles the storage and retrieval of data from physical storage.
- **Query Processor** : Interprets user queries, optimizes and executes them.
- **Transaction Manager** : Manages transactions and ensures ACID properties.
- **Security Manager** : Controls access, user authentication and authorization.
- **Backup & Recovery Manager** : Handles backup of data and recovery in case of failure.

★ DBMS Internal Architecture



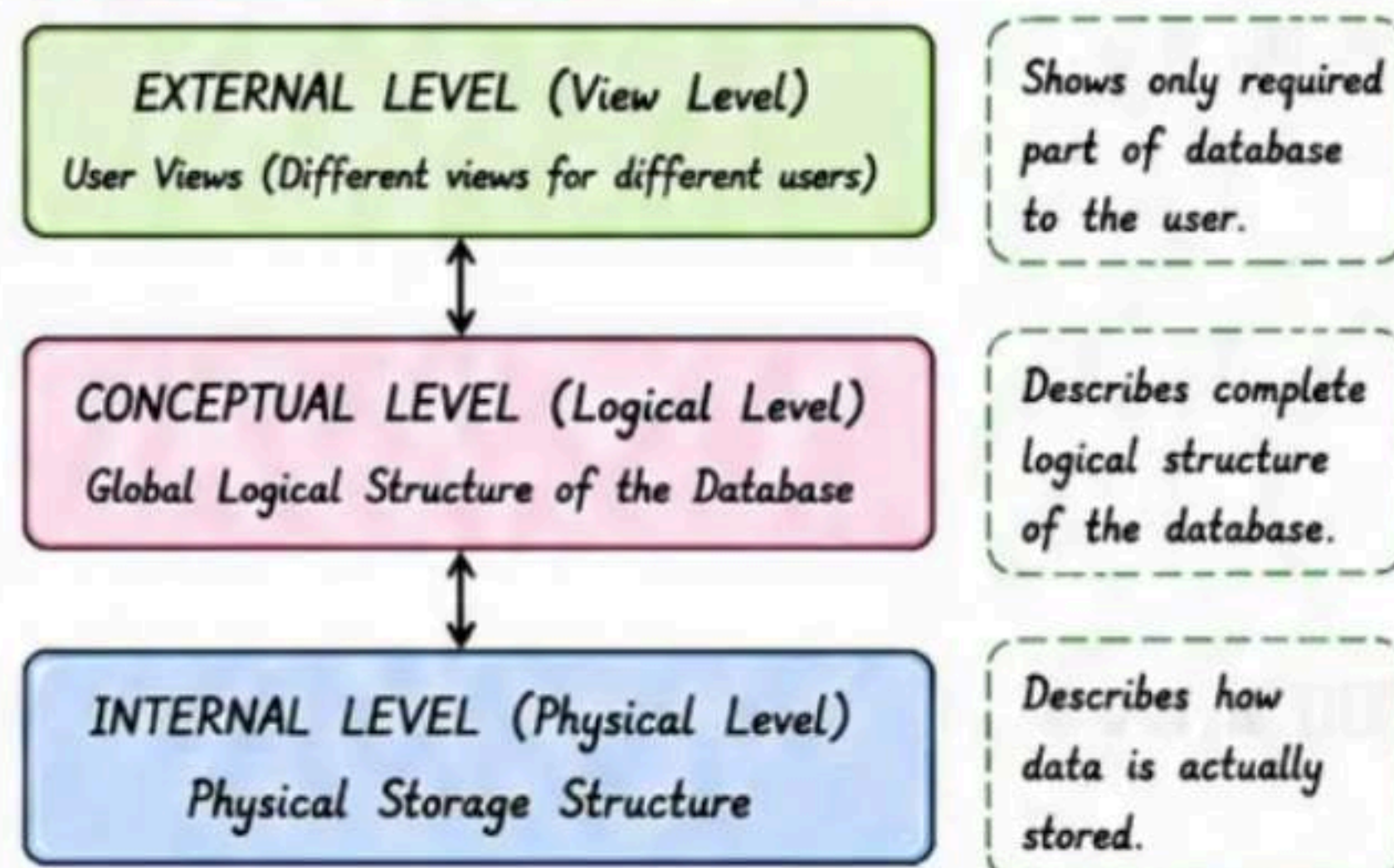
Remember !

DBMS is an interface between users/applications and the database. It manages data efficiently and ensures security, consistency and reliability.

★ ACID Properties of Transactions

Property	Description
Atomicity	All operations of a transaction are treated as a single unit. If any operation fails, the entire transaction is rolled back.
Consistency	A transaction brings the database from one consistent state to another consistent state.
Isolation	Concurrent transactions do not interfere with each other. Each transaction is unaware of others.
Durability	Once a transaction is committed, its changes are permanent even in case of system failure.

★ Three Level Architecture (ANSI / SPARC Model)



★ Database Languages

Language	Full Form	Purpose	Examples
DDL	Data Definition Language	Defines and modifies database structure (schema).	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	Used to insert, update, delete and retrieve data.	INSERT, UPDATE, DELETE, SELECT
DCL	Data Control Language	Controls access and permissions of the database.	GRANT, REVOKE
TCL	Transaction Control Language	Manages transactions and ensures data consistency.	COMMIT, ROLLBACK, SAVEPOINT
VDL	View Definition Language	Used to create and manage views.	CREATE VIEW, DROP VIEW

★ Database vs File System

Feature	File System	DBMS
Data Redundancy	High	Low
Data Consistency	Difficult to maintain	Easy to maintain
Data Integrity	Hard to ensure	Ensured using constraints
Data Security	Weak	Strong
Data Sharing	Limited	High
Data Independence	Not available	Logical & Physical Independence
Backup & Recovery	Difficult	Easy and automatic
Query Capability	Limited	Powerful (SQL)
Data Access	Program dependent	Data independent
Scalability	Difficult	Easy

★ DBMS Functions



Data Storage : Stores data efficiently and organizes it.



Data Retrieval : Allows fast and efficient retrieval of data.



Data Security : Protects data from unauthorized access and corruption.



Data Sharing : Allows multiple users to access and share data.



Backup & Recovery : Provides mechanisms to recover data in case of failure.

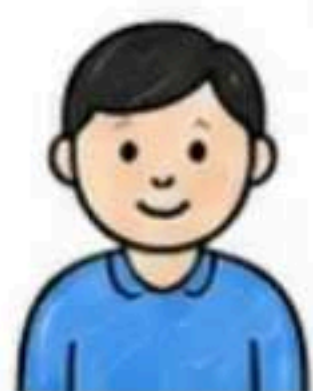


Concurrency Control : Manages simultaneous access by multiple users.



Integrity Enforcement : Ensures accuracy and reliability of data.

★ Database Administrator (DBA)



DBA

- Define and manage database schema.
- Handle user authorization and permissions.
- Ensure data security and integrity.
- Monitor performance and optimize queries.
- Perform backup and recovery.
- Handle storage management.
- Ensure high availability and concurrency.
- Maintain the database system.



Remember !

DBMS is not just a collection of data, it is a complete system that helps us store, manage, retrieve and protect data efficiently.

★ Examples of DBMS



Open source relational database. Uses SQL.



Powerful commercial relational database management system.



Microsoft SQL Server

Relational DBMS developed by Microsoft.



PostgreSQL

Advanced open source relational database.



mongoDB

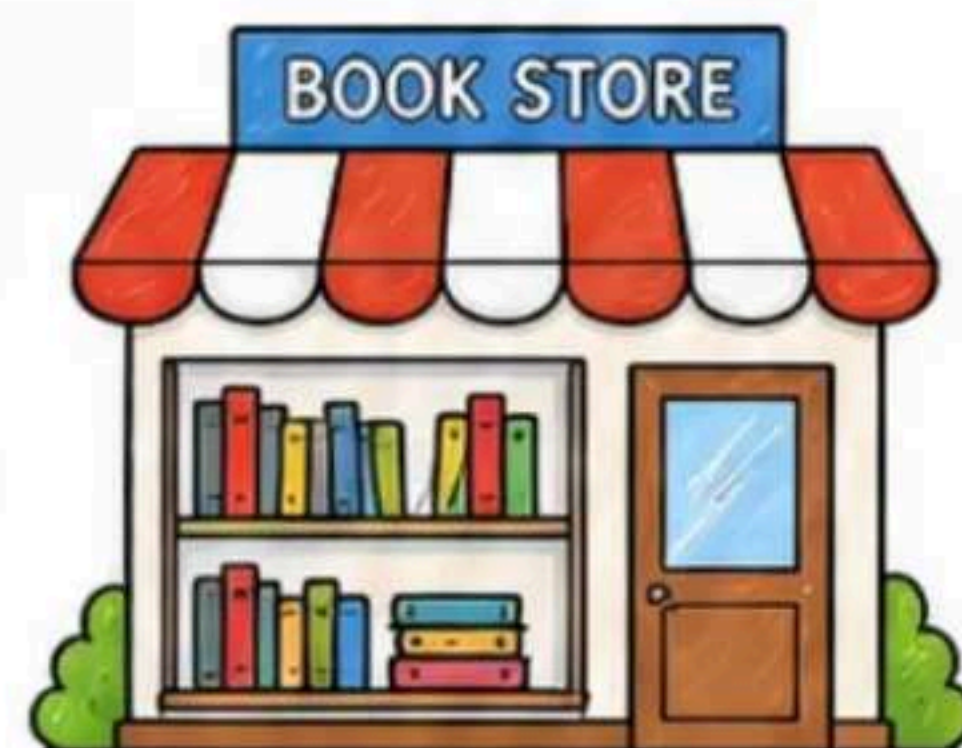
NoSQL database that stores data in flexible (JSON-like) documents.

★ DBMS Advantages

- ✓ Reduces data redundancy.
- ✓ Improves data consistency and integrity.
- ✓ Provides data security.
- ✓ Enables data sharing among multiple users.
- ✓ Provides backup and recovery.
- ✓ Offers data independence.
- ✓ Supports complex queries.
- ✓ Improves decision making.

Example

In a Book Store, DBMS is used to store information about books, customers, orders, sales, employees etc. It helps in managing all these data efficiently.



★ Introduction

DBMS architecture defines the structure and organization of a database system. It shows how different components interact with each other to provide data management services to the users.

★ DBMS Architecture (Three Level)

Three level architecture is also called ANSI / SPARC architecture.

Levels of Architecture

1. External Level (View Level)
 2. Conceptual Level (Logical Level)
 3. Internal Level (Physical Level)
- } Data Independence

★ Data Independence

- **Physical Data Independence** : Changes in internal level (physical storage) do not affect conceptual or external level.
- **Logical Data Independence** : Changes in conceptual level do not affect external views or application programs.

★ Data Abstraction

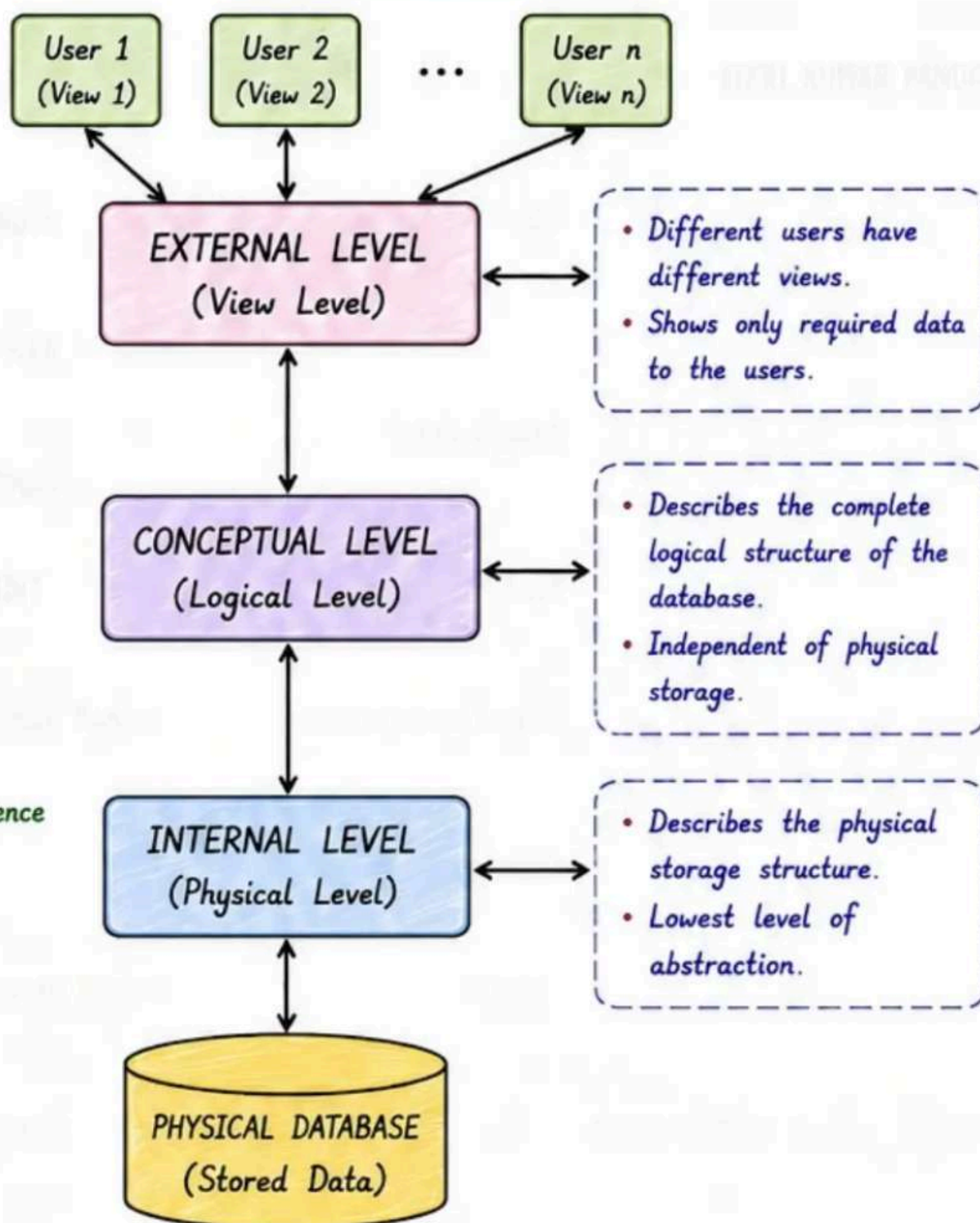
Data abstraction is the process of hiding unnecessary details and showing only essential information to the users.

Level	Abstraction	Hides	Shown To
External (View Level)	Shows only required part of data.	Rest of the database and complex details.	End users / Application Programs
Conceptual (Logical Level)	Shows logical structure of entire database.	Physical storage details and implementation.	Database Designer / Administrator
Internal (Physical Level)	Shows how data is actually stored physically.	Logical structure, relationships and constraints.	System Designer / Hardware

★ Working of Three Level Architecture

- Users send requests to the system.
- External level provides required view to the user.
- Conceptual level processes the request using logical structure.
- Internal level interacts with the physical database and retrieves the data.
- Data is returned back to the user through the same

Three Level Architecture



Note

Data abstraction helps in:

- ✓ Reducing complexity
- ✓ Improving security
- ✓ Avoiding data redundancy
- ✓ Easier maintenance
- ✓ Better data management

Remember !



- External level deals with "What" the user sees.
- Conceptual level deals with "What" the entire database contains.
- Internal level deals with "How" the data is actually stored.
- Three level architecture provides data independence

★ Components of Three Level Architecture

1. External Level (View Level)

Contains multiple user views.

- Each user has his own view of data.

2. Conceptual Level (Liogical Level)

Represents the global logical structure of the entire database.

- Independent of physical storage.

3. Internal Level (Physical Level)

Describes how data is stored in the storage devices.

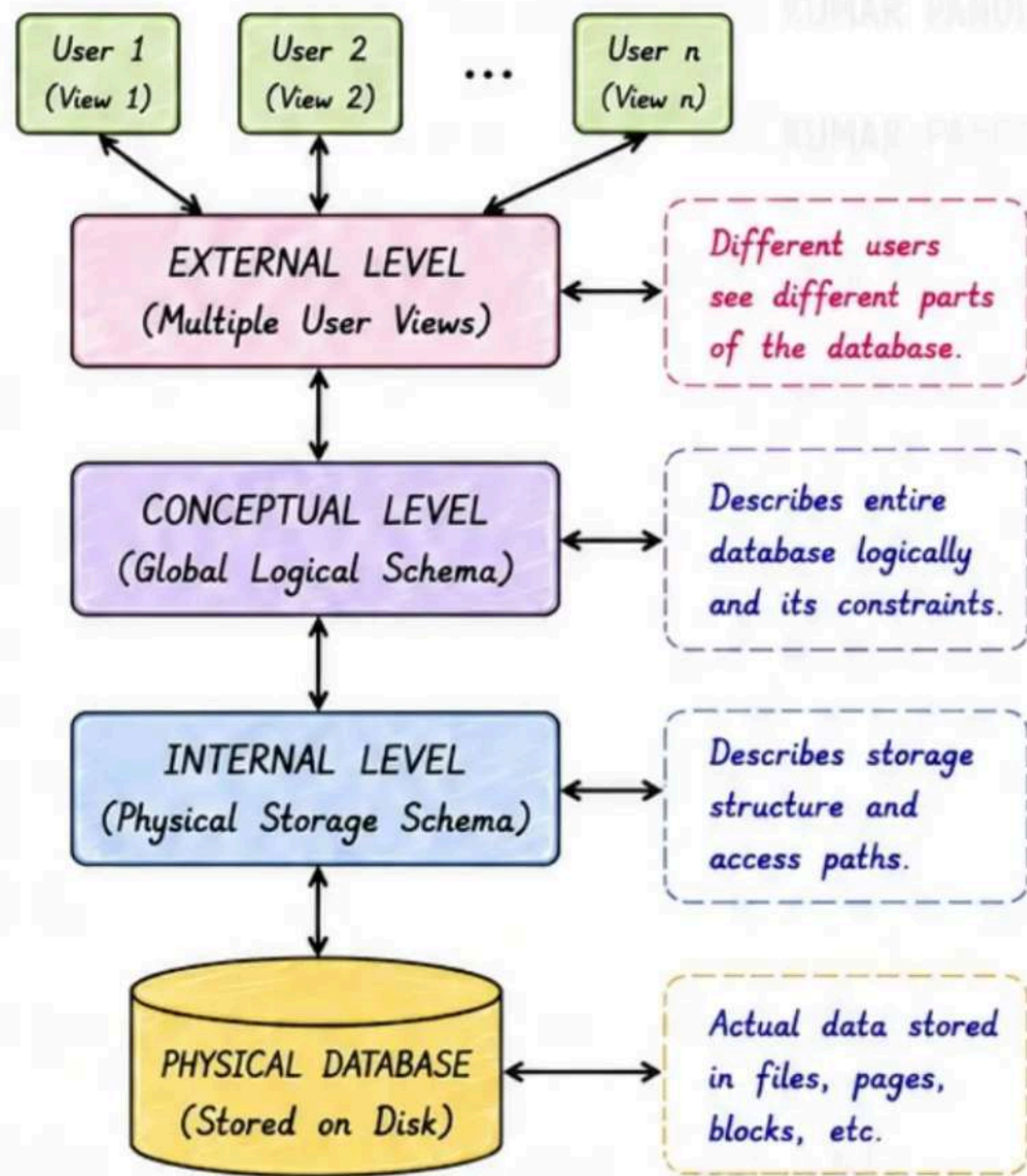
- Independent of logical structure.

Key Point

- Upper level views are dependent on lower level.
- Lower level changes do not affect upper level.
- This architecture provides physical data independence.



Architecture Diagram



★ Mapping Between the Levels

External / Conceptual Mapping

Maps user views to the conceptual schema.

It defines how each external view is related to the central conceptual view.

Conceptual / Internal Mapping

Maps conceptual schema to internal (physical) schema.

It defines how data is stored physically on storage devices.

External / Internal Mapping

Provides a direct relationship between user views and storage.

Helps to retrieve data without knowing physical details.

★ Data Independence in Three Level Architecture

Type	Definition	Example	Benefit
Physical Data Independence	Changes in the internal level (physical storage) do not affect conceptual or external levels.	Changing file organization or adding an index does not affect user programs.	✓ Easier maintenance ✓ System flexibility
Logical Data Independence	Changes in the conceptual level (logical structure) do not affect external views.	Adding a new attribute in the database does not affect existing user views.	✓ Application flexibility ✓ Easier modification

★ Advantages of Three Level Architecture

- ✓ Provides data independence.
- ✓ Improves security by restricting access.

Example to Understand



Different people in a bank see data differently.

Chapter 2 : DBMS ARCHITECTURE & DATA ABSTRACTION

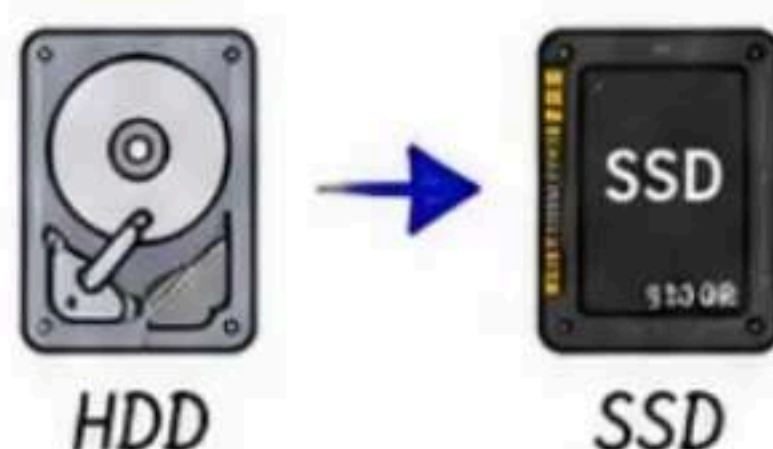
★ Data Independence (Explained with Example)

Data independence allows changes at one level of the architecture without affecting the other level.

A. Physical Data Independence

Changes in physical storage do not affect the conceptual or external level.

Example : If we change the storage device from HDD to SSD, the logical structure and user views remain the same.



Key Points

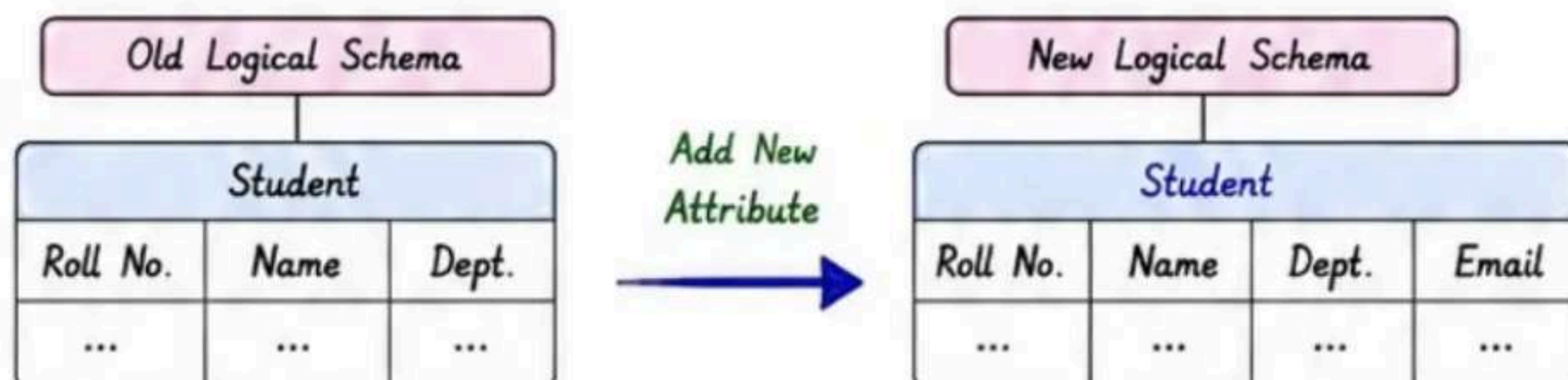
- Physical level changes (e.g., file organization, indexing, storage devices) do not affect logical level.
- Logical level changes (e.g., adding a new table) do not affect user views.
- Ensures flexibility and easier maintenance of the database system.



B. Logical Data Independence

Changes in conceptual (logical) schema do not affect external level (user views).

Example : If a new attribute is added to a table, existing user applications that do not use this attribute will continue to work without modification.

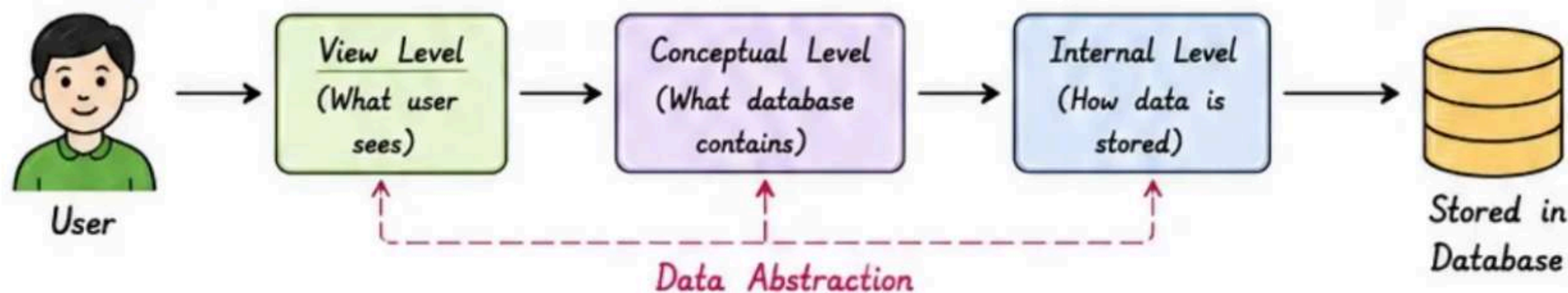


Note

Data independence is one of the biggest advantages of the DBMS three level architecture.

★ Data Abstraction

Data abstraction means showing only necessary data to the user and hiding unnecessary details.



★ Advantages of Data Abstraction

- Hides complex details from users.
- Improves security by restricting direct access to data.
- Makes database system easier to understand and use.
- Helps in database maintenance and evolution.

Note

Data abstraction helps in providing

Example



★ Concurrency Control in DBMS

Concurrency control ensures that multiple transactions can execute simultaneously without causing data inconsistency.

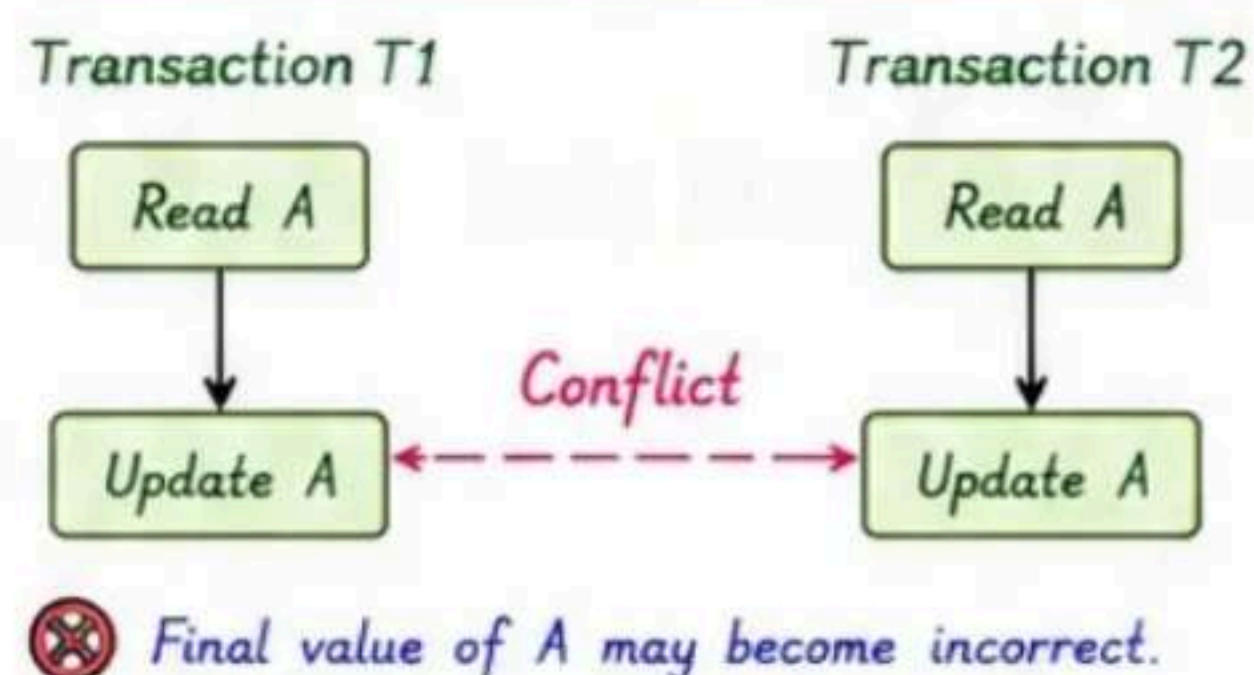
Goals

- Ensure consistency of data.
- Prevent conflicts between concurrent transactions.
- Maintain isolation and correctness.

Concurrency Control Techniques

Technique	Description	Properties
Locking	Locks are used to control access to data items. Shared lock (read) allows read, Exclusive lock (write) allows write.	• Simple and widely used • Can cause deadlock
Timestamp Ordering	Transactions are assigned timestamps. Operations are performed in timestamp order.	• Ensures serializability • May cause rollback of transactions
Multiversion Concurrency Control (MVCC)	Multiple versions of data are maintained. Transactions read from appropriate versions.	• Allows more concurrency • More storage required
Optimistic Concurrency Control	Transactions run without locking. Conflicts are checked at commit time.	• Good for read-mostly systems • Costly if conflicts are frequent

Problem Without Concurrency Control

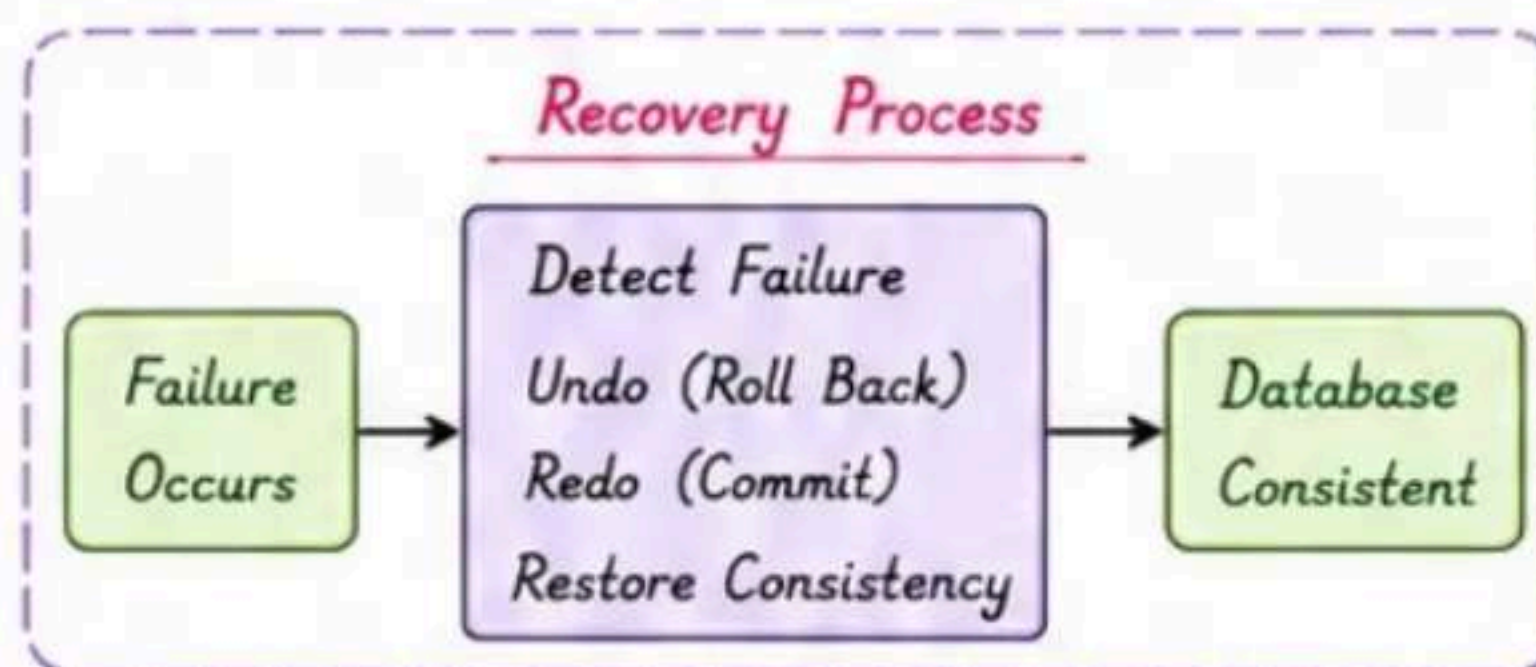


★ Recovery Management

Recovery ensures that the database returns to a consistent state after failures.

Key Objectives

- Detect failures.
- Undo incomplete transactions.
- Redo committed transactions.
- Ensure database consistency.



Common Failures

- Transaction failure
- System crash
- Media (disk) failure

★ Database Utilities

These utilities help in database maintenance and performance.

Backup

Creates copies of database for recovery.



Restore

Restores database from a backup in case of failure.



Reorganization

Improves database performance by reorganizing data.



★ Security Management

Protects data from unauthorized access and ensures confidentiality, integrity and availability.

Security Measures

- **Authentication** - Verifies user identity
- **Authorization** - Controls access to data
- **Encryption** - Protects data during storage and transmission
- **Auditing** - Monitors user activities and detects misuse

Benefits



- Prevents data theft
- Ensures data integrity
- Maintains system reliability

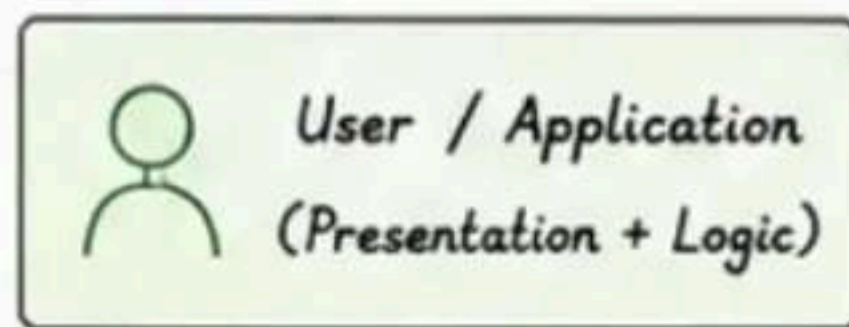
★ DBMS ARCHITECTURES : 1-TIER, 2-TIER & 3-TIER

DBMS can be deployed in different architectures depending on how clients, application and database are arranged and connected.

Key Points

- ★ Tiers show how many layers exist between user and data.
- ★ As number of tiers increases, scalability, security and maintenance improve.

★ 1-TIER ARCHITECTURE



Working :

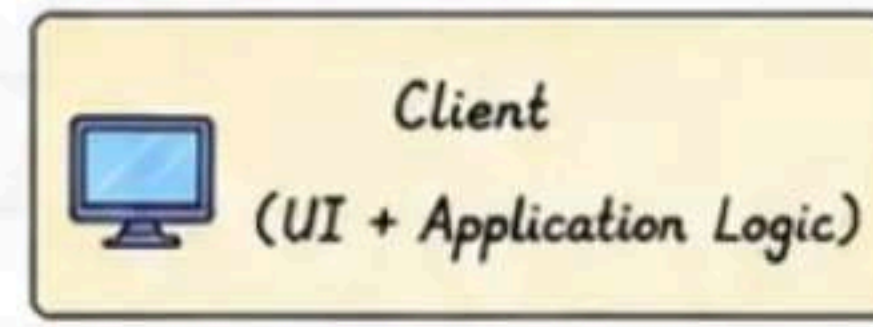
User and DBMS reside on the same system. Application directly accesses the database.

Example :

Personal system, standalone desktop application.

- + Simple and fast
- Not suitable for multi-user environment

★ 2-TIER ARCHITECTURE



Working :

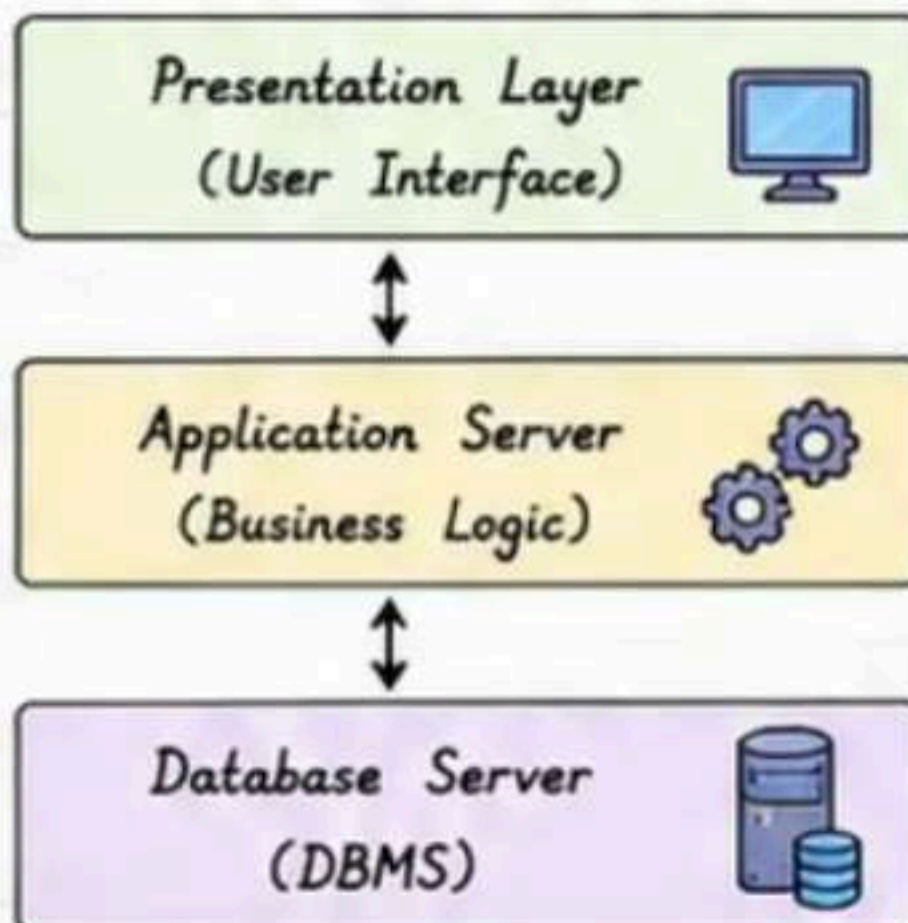
Client (application) communicates with Database Server over network.

Example :

Library management system, banking application.

- + Easy to develop and maintain
- Limited scalability, business logic on client side

★ 3-TIER ARCHITECTURE



Working :

User interacts with presentation layer. Requests go to application server which processes business logic and interacts with database server.

Example :

E-commerce websites, online banking systems, enterprise applications.

- + Highly scalable
- + Better security
- + Easy maintenance
- More complex
- Cost of setup is high

★ Comparison : 1-Tier vs 2-Tier vs 3-Tier

Feature	1-Tier Architecture	2-Tier Architecture	3-Tier Architecture
Structure	User, application and DBMS on same system.	Client communicates directly with database server.	Presentation, application and database server in separate tiers.
Client	No separate client.	Client contains UI + some business logic.	Client only contains UI.
Application Logic	On same system with DBMS.	On client side.	On application server.
Scalability	Very poor	Limited	High
Security	Low	Moderate	High
Maintenance	Difficult for large system	Moderate	Easy
Performance	High for small system	Moderate	High (for large system)
Example	Personal DB, Small desktop app	Library system, Payroll system	Online banking, ERP, E-commerce
Cost	Low	Medium	High

★ Advantages of Multi-Tier Architecture

- Better scalability and reliability
- Enhanced security

★ Limitations

- More complex system design

★ Use Cases

- 1-Tier : Small systems, personal use

A data model is a collection of conceptual tools for describing data, data relationships, data semantics and consistency constraints.

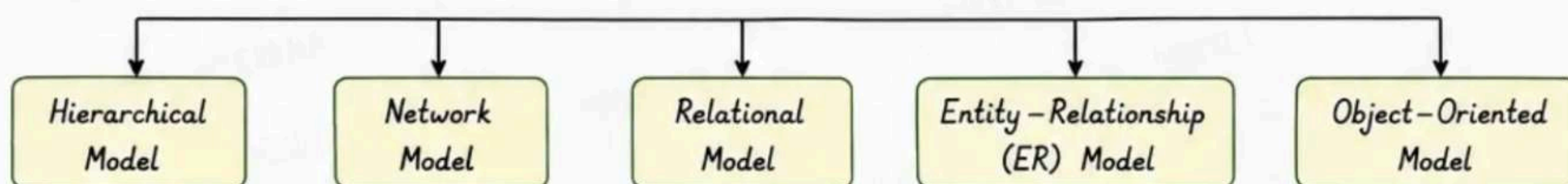
★ Purpose of Data Models

- Provide a way to represent real world data.
- Support data abstraction.
- Help in database design.
- Ensure data integrity and consistency.

Components of Data Model

- ★ Data Structures – How data is organized
- ★ Data Operations – Operations on data
- ★ Data Integrity Rules – Constraints on data

★ Types of Data Models (Overview)

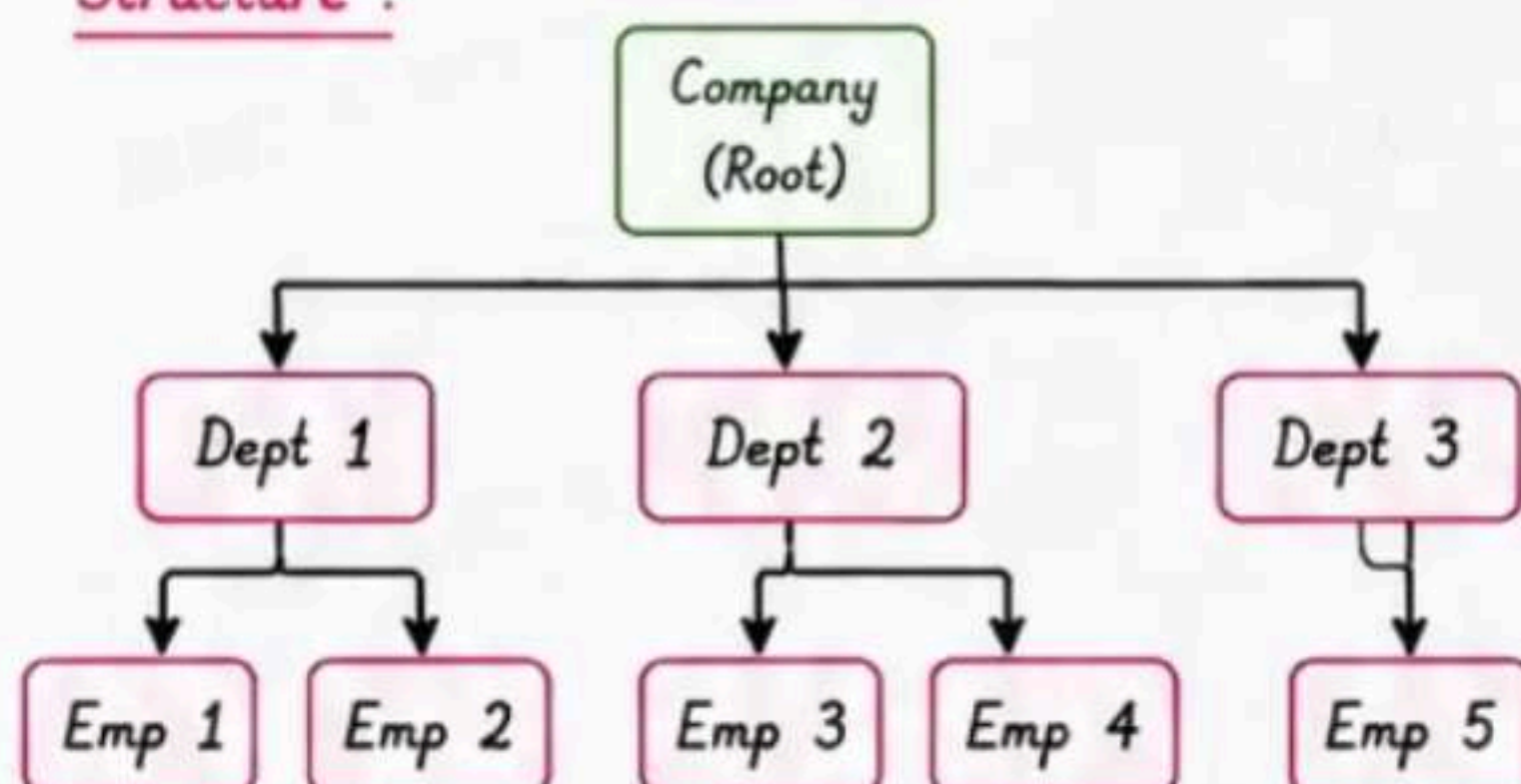


★ 1. Hierarchical Model

Definition :

Data is organized in a tree-like structure. There is one root and each child has only one parent.

Structure :



Example :

Organization → Departments → Employees

Characteristics :

- Each child has only one parent.
- One-to-many relationship.
- Navigation is top-down.
- Simple and efficient.

Advantages :

- + Simple to understand.
- + Efficient for hierarchical data.
- + Data integrity is easy to maintain.

Limitations :

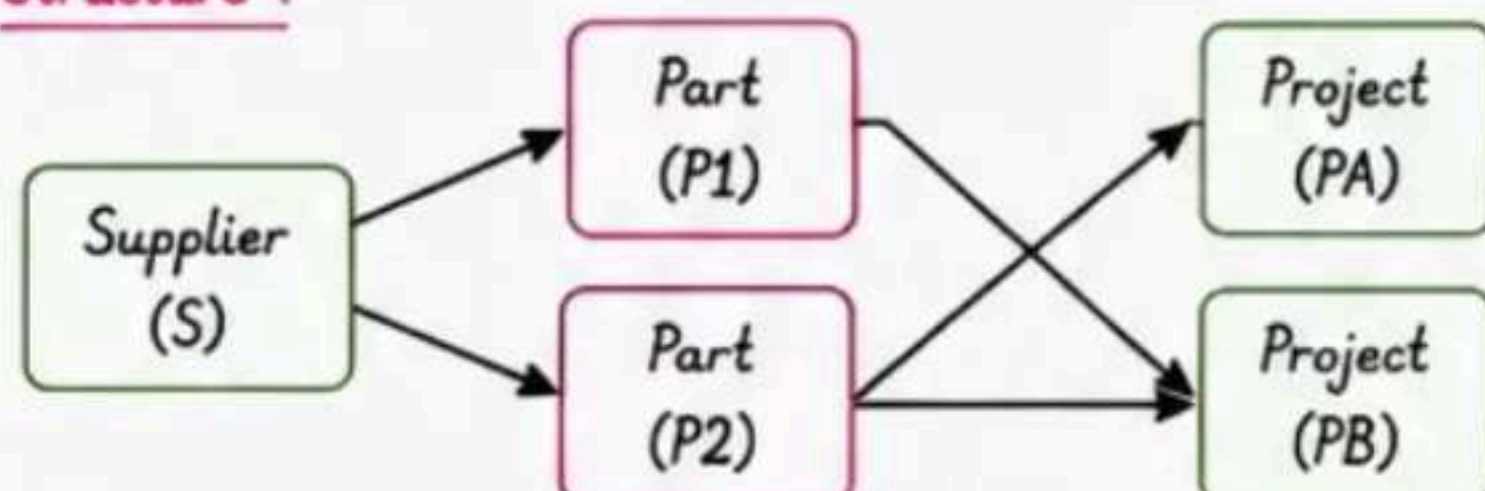
- Inflexible structure.
- Difficult to represent many-to-many relationships.
- Data redundancy may occur.

★ 2. Network Model

Definition :

Data is organized as a graph. A record can have multiple parent and multiple child records.

Structure :



Example :

Characteristics :

- Records are linked using pointers.
- Many-to-many relationships supported.
- More flexible than hierarchical model.

Advantages :

- + Supports complex relationships.
- + Less data redundancy.
- + More flexible structure.

Limitations :

★ 3. Relational Model

Page 16

Definition :

Data is organized in the form of tables (relations). Each table consists of rows (tuples) and columns (attributes).

Structure :

Relation (Table) : EMPLOYEE

EmpID	EmpName	Dept	Salary
101	Rohit	IT	50000
102	Neha	HR	45000
103	Amit	Finance	48000
104	Pooja	IT	52000

Tuples (Rows) are indicated by a bracket on the left side of the table rows.

Attributes (Columns) are indicated by a bracket below the table columns.

Example :

EMPLOYEE(EmpID, EmpName, Dept, Salary)

Characteristics :

- Data is represented in tables.
- Each column has unique name.
- Each row is unique.
- Relationships are established using keys.

Advantages :

- + Simple and easy to understand.
- + Data independence.
- + Supports ad-hoc queries easily.
- + Integrity constraints can be enforced.

Limitations :

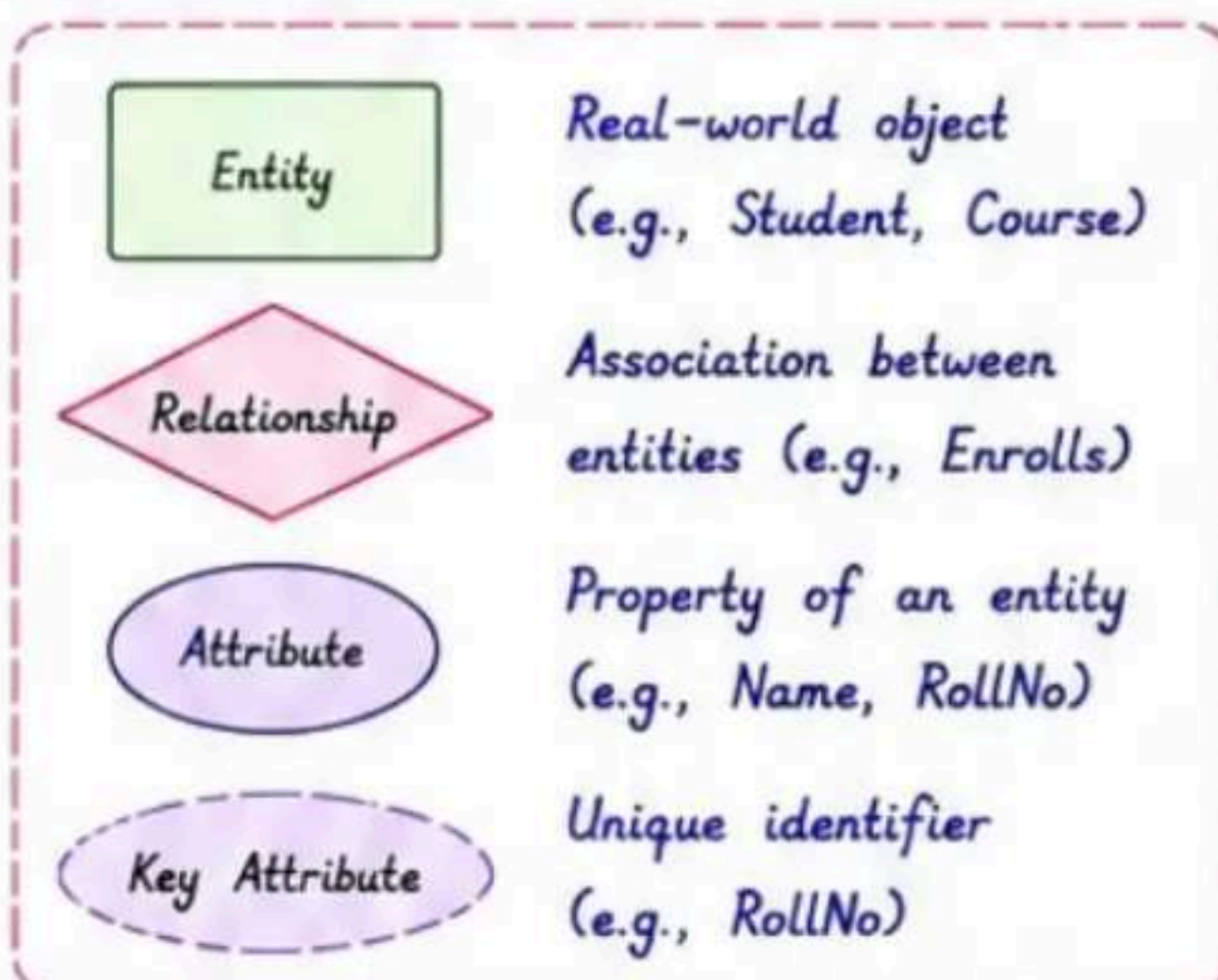
- Not suitable for complex relationships in some cases.
- Performance may degrade with very large data.
- Requires more storage.

★ 4. Entity-Relationship (ER) Model

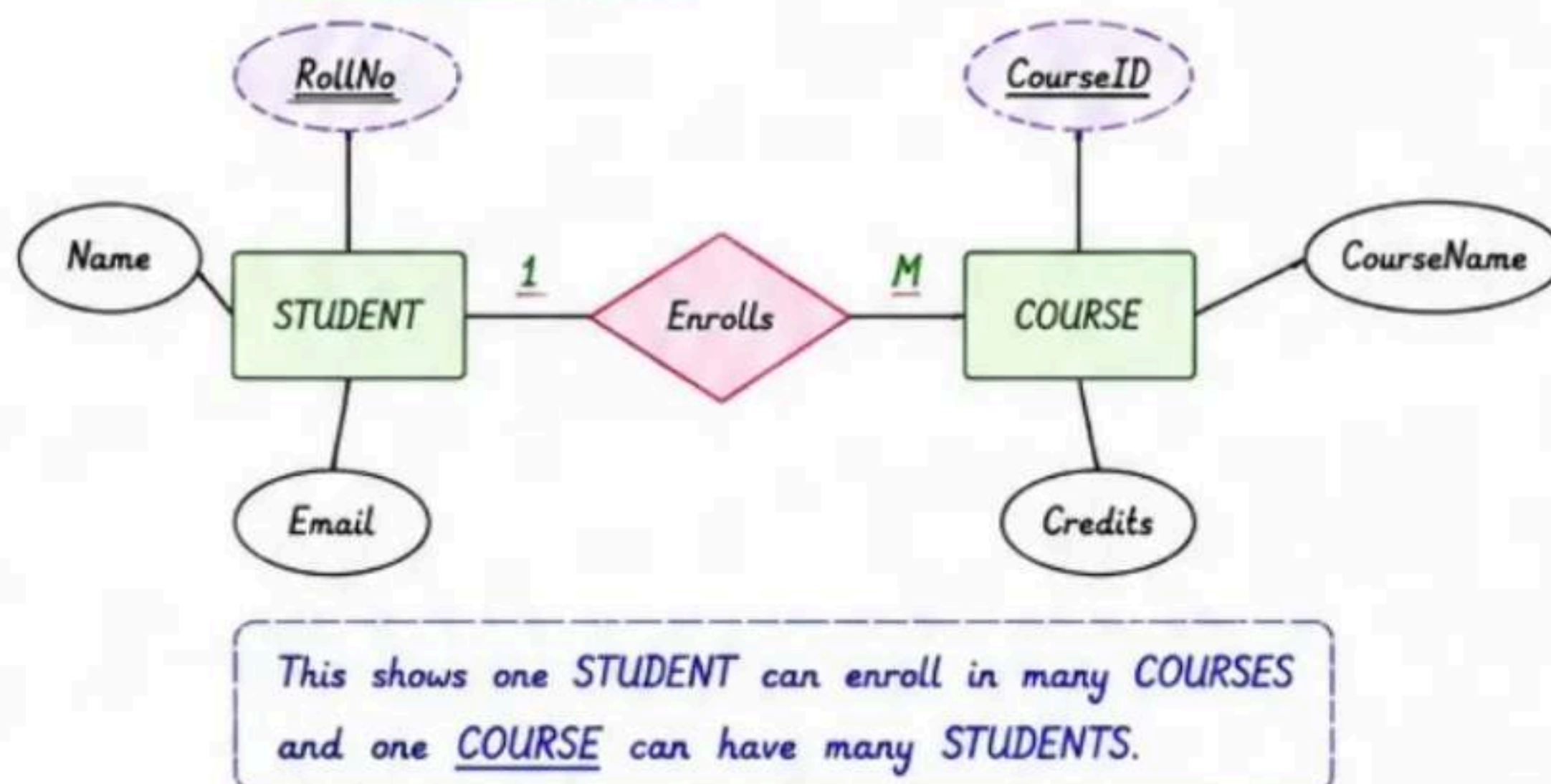
Definition :

ER Model is a high-level conceptual data model used to represent real-world entities and their relationships.

Components :



ER Diagram Example :



Advantages :

- + Easy to understand.
- + Independent of any DBMS.
- + Helps in database design.

Limitations :

- Does not show how data is stored.
- Complex for large systems.
- Needs conversion to relational model.

Example :

A university database can be designed using ER model to represent Students, Courses, Teachers, Departments etc.

★ 5. Object-Oriented Model

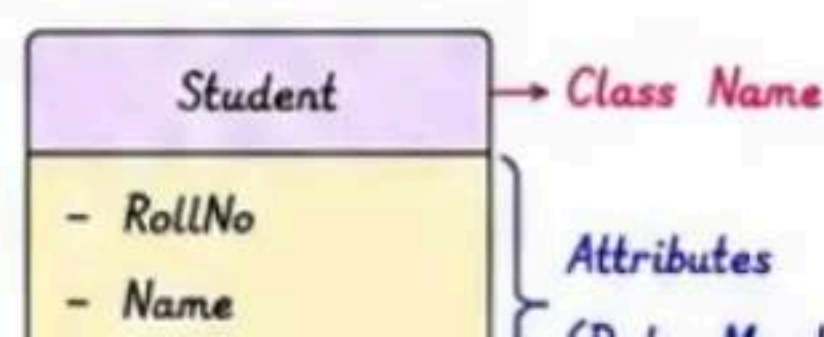
Definition :

Data is represented as objects. An object is a combination of data (attributes) and methods (operations).

Key Concepts :

- Object : Real-world entity with identity.

Example :



Advantages :

- + Real-world representation.
- + Supports inheritance.

Limitations :

- Complex to implement.
- Not widely supported by

Feature	Hierarchical Model	Network Model	Relational Model	ER Model	Object-Oriented Model
Data Structure	Tree structure (Parent → Child)	Graph structure (Many-to-many)	Table (Rows & Columns)	Entities, Relationships and Attributes	Objects (Data + Methods)
Relationships	One-to-many only	Many-to-many supported	Through Keys (Primary & Foreign)	One-to-one, One-to-many, Many-to-many	Through object references
Data Handling	Rigid and less flexible	More flexible than hierarchical	Very easy and structured	High-level conceptual representation	Very flexible and real-world like
Data Independence	Low	Better than hierarchical	High (Physical & Logical)	High (Conceptual level model)	High
Ease of Use	Difficult	Complex	Easy (SQL support)	Easy to understand (Conceptual)	Moderate to Complex
Scalability	Poor	Moderate	High	High (during design phase)	High
Redundancy	High	Less	Very less	Less	Less
Example	File system directory	Parts in a manufacturing company	Banking, Payroll, Library system	University database, Hospital system	CAD systems, Multimedia systems

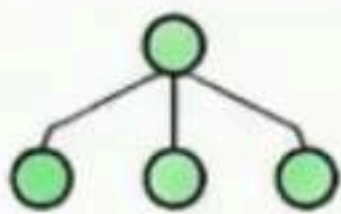
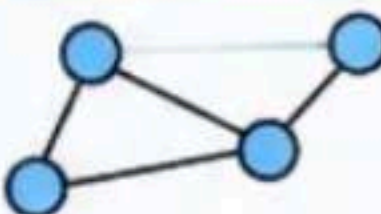
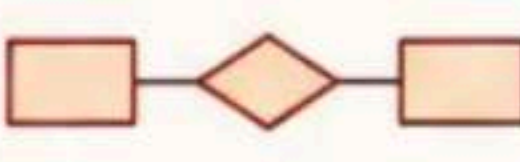
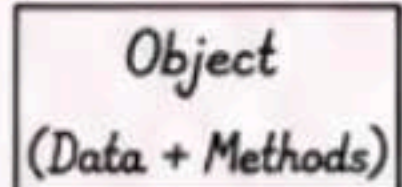
★ Advantages of Modern Data Models

- ★ Provide a structured way to organize and manage data.
- ★ Reduce data redundancy and inconsistency.
- ★ Improve data integrity and accuracy.
- ★ Support efficient data retrieval and manipulation.
- ★ Help in designing scalable and secure databases.
- ★ Facilitate data abstraction and data independence.

★ Limitations of Data Models

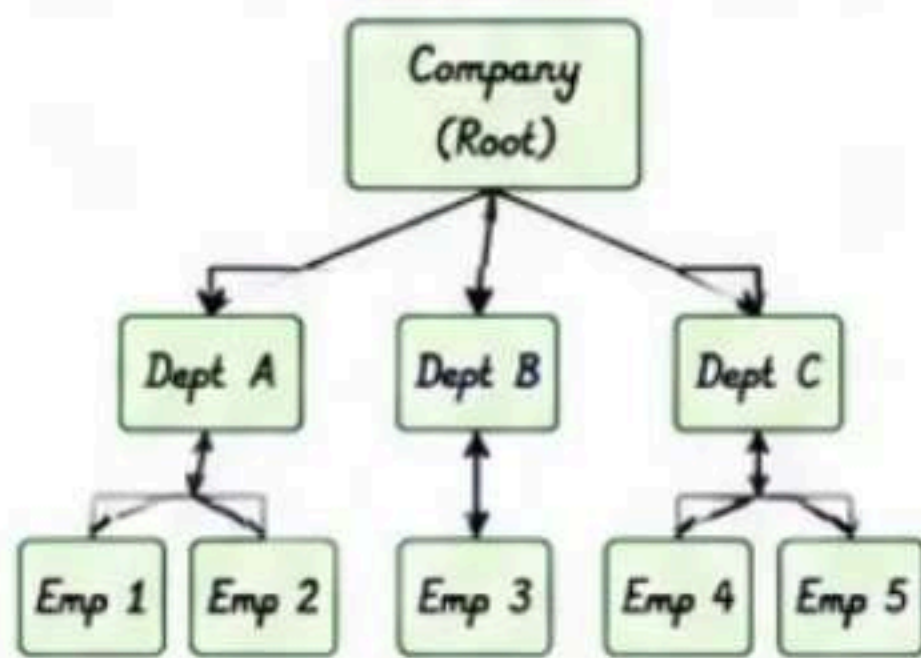
- Some models are complex to design and implement.
- Conversion from one model to another is difficult.
- Performance may degrade with very large data.
- Object-Oriented model is not widely supported.
- Requires skilled designer and understanding.

★ Use Cases / Real-World Examples

	Hierarchical Model	→ File system in computers, organization structure (Company → Departments → Employees).
	Network Model	→ Many-to-many relationships like Suppliers → Parts → Projects, Telecommunication networks.
	Relational Model	→ Banking systems, Library management, E-commerce, Hospital management, Payroll systems.
	ER Model	→ Database design for University, Hospital, School, Insurance, Online reservation system.
	Object-Oriented Model	→ CAD/CAM systems, Multimedia databases, Gaming applications, AI and simulation systems.

★ Summary

★ Detailed View of Each Model



Hierarchical Model

Data is organized in a tree structure. Each child has only one parent.

Advantages

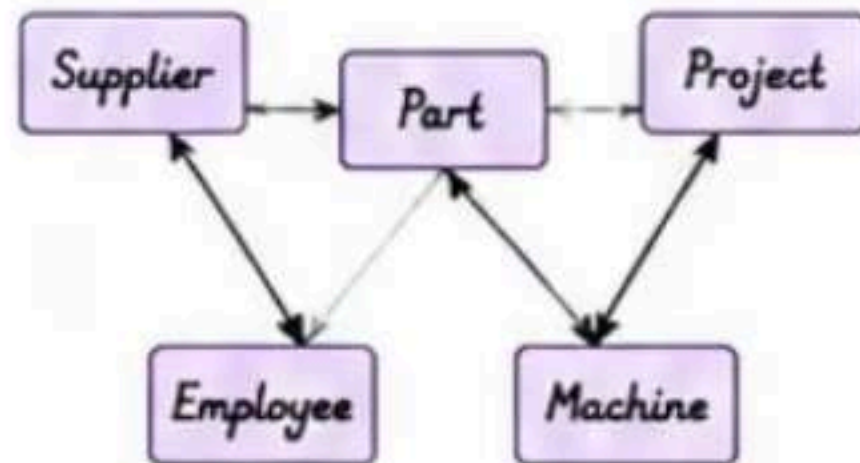
- Simple structure
- Easy to understand
- Efficient for hierarchical data
- Data integrity is easy to maintain

Limitations

- Inflexible structure
- Difficult to represent many-to-many relationships
- Data redundancy may occur

When to Use

When data has natural hierarchical relationship.
(Example: Organization structure)



Network Model

Data is organized as a graph where a record can have multiple parent and multiple child records.

Advantages

- Supports many-to-many relationships
- More flexible than hierarchical
- Better data independence
- Less redundancy

Limitations

- Complex to design and maintain
- Navigation is difficult
- Integrity constraints are complex

When to Use

When data has complex many-to-many relationships.
(Example: Telecom network)

Relational Model

EmpID	EmpName	Dept	Salary
101	Rohit	IT	50000
102	Neha	HR	45000
103	Amit	Finance	48000
104	Pooja	IT	52000
...			

Data is organized in tables (rows and columns). Relationships are established using keys.

Advantages

- Simple and easy to use
- High data independence
- Very easy to query (SQL)
- Reduces data redundancy
- Integrity constraints can be enforced

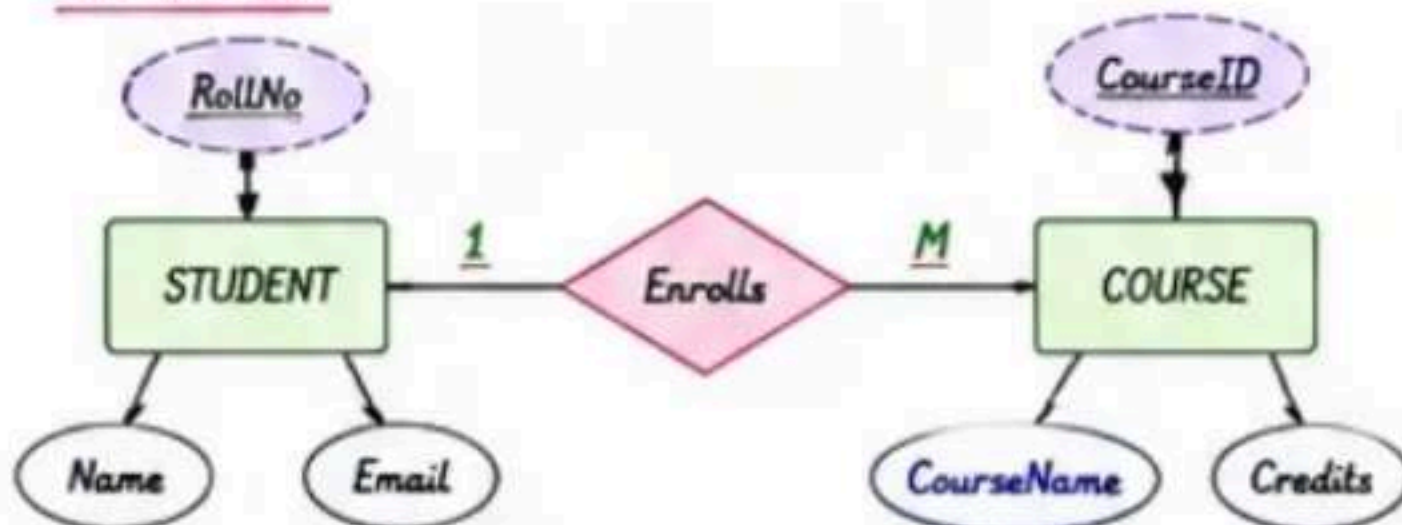
Limitations

- Not suitable for complex data types
- Performance may degrade with very large data
- Requires more storage

When to Use

Most general purpose applications.
(Example: Banking, Payroll, Library Management)

ER Model



ER model is a high-level conceptual model used to represent real-world entities and relationships.

Advantages

- Easy to understand
- Helps in database design
- Maps well to relational model
- Independent of any DBMS

Limitations

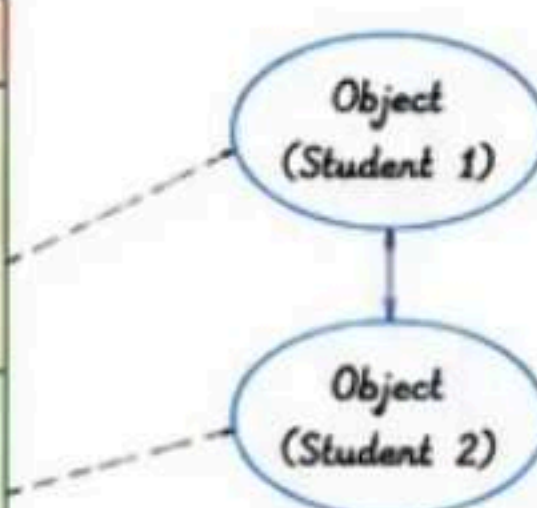
- Does not show how data is stored
- Needs conversion to relational model

When to Use

During requirements analysis and conceptual design phase.
(Example: University Database Design)

Object-Oriented Model

Class: Student
Attributes (Data)
- RollNo
- Name
- Course
Methods (Functions)
+ display()
+ getDetails()



Data is represented as objects which combine data (attributes) and methods (functions).

Advantages

- Real-world representation
- Supports inheritance
- Data hiding (security)
- Code reusability

Limitations

- Complex to implement
- Not widely supported by all DBMS
- More storage required

When to Use

For complex applications like CAD/CAM, Multimedia, AI and Simulation systems.

★ Quick Summary

Model	Best For	Relationship Support	Complexity	Performance	Flexibility
Hierarchical	Tree structured data	One-to-many	Low	High (for simple data)	Low
Network	Complex relationships	Many-to-many	High	Good	High
Relational	General purpose	One-to-many, Many-to-many	Medium	High (with indexing)	High

The **ER Model (Entity-Relationship Model)** is a high-level conceptual data model used to represent real-world data and their relationships in a simple and understandable way.

Key Points

- ★ ER Model is used in requirement analysis and database design.
- ★ It is implementation independent.
- ★ It helps to represent data in a graphical form.

★ 1. Introduction to ER Model

ER Model represents real-world **entities** (objects), their **attributes** (properties) and the **relationships** among them.

Uses :

- Requirement collection and analysis
- Database conceptual design
- Communication between users and designers
- Basis for logical database design

★ 2. Entity and Entity Set

- **Entity** : Any real-world object that can be uniquely identified is called an entity.

Example : Student, Course, Teacher, Department, Book

- **Entity Set** : A collection of similar type of entities is called an entity set.

Example : SET OF ALL STUDENTS, SET OF ALL COURSES

★ 3. Basic ER Symbols (Notation)

Symbol	Name	Description	Example
	Entity	Represents a real-world object.	STUDENT
	Relationship	Represents association between entities.	ENROLLS
	Attribute	Represents property or characteristic of an entity.	Name
	Key Attribute	Attribute that uniquely identifies an entity.	RollNo
	Weak Entity	Entity that cannot be identified by its attributes alone.	DEPENDENT
	Identifying Relationship	Relationship used to identify weak entity.	BELONGS TO

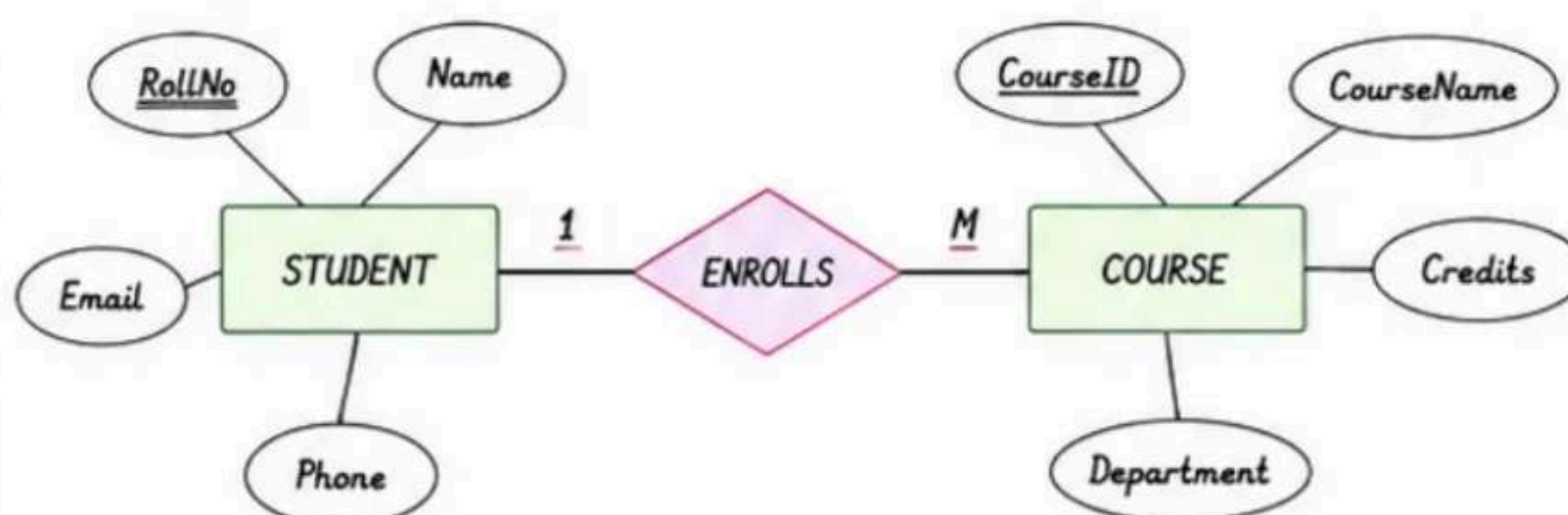
Other Notations

- Line : Connection between entity and relationship
- == Total Participation (Mandatory)
- Partial Participation (Optional)
- 1 One
- N Many
- M Many (another set)

★ 4. Simple ER Diagram Example

Example : Student-Course ER Diagram

- A student can enroll in many courses.
- A course can have many students.
- Each student is identified by RollNo.
- Each course is identified by CourseID.
- Relationship between them is ENROLLS.



Summary

- ✓ Entity represents an object in real world.
- ✓ Entity Set is a collection of similar entities.
- ✓ Attributes describe the properties of entities.
- ✓ Relationship shows association between entities.
- ✓ ER Diagram uses standard symbols for easy understanding.

Why ER Model is Important ?

- ★ Easy to understand for all users.
- ★ Helps in better communication.
- ★ Serves as blueprint for database.
- ★ Reduces design errors.
- ★ Improves data consistency.

Remember

- Entity = Noun (Object)
- Attribute = Property
- Relationship = Verb (Action)
- ER Diagram = Visual Representation